Experimental Methodology — Commentary

Lesson 5 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

Not examinable · reading time about 2 hours

Contents

Slide 1 — Lesson 5: Experimental Methodology 5

Part I — A score is a measurement **5**
Slide 2 — Before we start: lesson 4, concluded 5
Slide 3 — We broke a promise on purpose 6
Question — so why did lesson 4 do it at all, if it was forbidden? 6
Slide 4 — Today: how to tell whether it works 6
Question — the word "bias" seems to mean two things today. Does it? 6
Slide 5 — A test score is a measurement 6
Question — unbiased estimate of what, exactly? 6
Question — what does unbiased mean, if two correct splits give different scores? . . 7
Slide 6 — The fleet, cut down 7
Question — why cut the fleet to 800? Is the demonstration rigged? 7
Slide 7 — 200 honest splits, one dataset 7
Question — which of the 200 values is the true score? 8
Question — why does the notes call the right-hand tail the dangerous one? 8
Slide 8 — Why do the 200 scores spread so widely? 8
Question — why does one failing drive moving change the AUC by 0.015? 8
Question — "each test set holds 7% failures", right? 8
Worked example — the AUC as a count of pairs 9
Slide 9 — Parameters and hyperparameters 9
Question — so hyperparameters are the ones chosen before the fit? 9
Question — why is scikit-learn's C "the role of 1/ " and not simply 1/ ? 9
Slide 10 — Choosing a hyperparameter on a validation set: two recipes 10
Question — what is the validation set for, and how is it used? 10
Question — in step 3, which model wins? 10
Question — why does the left recipe need one fit per candidate and the right one a single fit? 11
Slide 11 — On each of slide 7's 200 splits, three models compete 11
Question — what does the third column count? 11
Question — C = 0.01 wins 89 times against 42: isn't it the better model? 11
Slide 12 — The same three models: who wins, and how they score 11
Question — what does "winner's AUC minus runner-up's AUC" mean? 12

Question — so the message is that there always seems to be a winner, but the models are equivalent? . . . 12

Part II — Cross-validation, and what it estimates **12**
Slide 13 — k-fold cross-validation: the validation set, rotated . . . 12
Question — does k-fold run on the validation set? . . . 13
Question — one model per fold, or one per round? . . . 13
Question — if the block is called validation, why does scikit-learn call it test? . . . 13
Slide 14 — What it actually estimates . . . 13
Question — so to measure a model I need no test set, and to tune one I do? . . . 13
Question — what model do I ship, then? . . . 13
Question — and if the score is not good enough, I change the recipe and cross-validate again? . . . 14
Question — why "slightly pessimistic"? . . . 14
Slide 15 — Five rounds, five different models; you ship a sixth, fitted on all 800 . . . 14
Question — why do the coefficients change, with the same algorithm and the same C? 14
Question — is the shipped model one of the five, or their average? . . . 15
Slide 16 — Report the spread, but not as a confidence interval . . . 15
Question — why can I not divide by $\sqrt{5}$? . . . 15
Question — the models train on overlapping data, so the scores are correlated? . . . 15
Worked example — five fold scores, their spread and the tempting interval . . . 16
Slide 17 — 300 new fleets: twice the naive spread; compare the widths . . . 16
Question — where do the 300 fleets come from? . . . 17
Question — the two bars are centred in different places. Which is right? . . . 17
Slide 18 — Stratify when a class is rare . . . 17
Question — must every fold have the same number of positives? . . . 17
Question — does stratification make the scores less spread? . . . 17
Slide 19 — Change only the seed, 40 times: how far does the score move? (optional) . . . 18
Question — why does the seed matter so much less for cross-validation? . . . 18
Question — so cross-validation is accurate to $\pm$ 0.003? . . . 18
Slide 20 — How many folds? (optional) . . . 18
Question — why are the scores more correlated with a larger k? . . . 19
Question — what is leave-one-out for? . . . 19
Question — is k a hyperparameter? . . . 19
Slide 21 — When the rows are not independent . . . 19
Question — how can the model recognise a drive it was never told the name of? . . . 19
Question — does every model fall for it? . . . 19
Slide 22 — Split by the thing you must generalise to . . . 19
Question — how do I pass the groups, and what about the test set? . . . 20
Question — do the groups have to be the same size? . . . 20
Question — what about data ordered in time? . . . 20
Slide 23 — GroupKFold, drawn: each drive in one fold . . . 21
Slide 24 — Notebook 1, live . . . 21
Slide 25 — Break . . . 21

Part III — What the error is made of **22**
Slide 26 — Error too high: more data, or another model? . . . 22
Question — is 6,064.3 measured on a validation set? . . . 22

Question — is it lesson 3's dataset? . . . 22
Question — why go through a lab at all, if cross-validation does the diagnosis? . . . 22
Slide 27 — A lab where we know the truth . . . 22
Question — so for degree 3 there are 300 models? . . . 22
Question — what are the 300 models for, if in real life I have one? . . . 22
Slide 28 — Three hundred parallel universes: one model each . . . 23
Question — how is the red curve obtained? . . . 23
Question — degree 1's lines fan out: isn't that bias? . . . 23
Question — at degree 12 the red curve shoots up at the right edge, though most lines do not. Why? . . . 23
Slide 29 — The bias–variance decomposition: expected error = bias² + variance + noise . 23
Question — what is the name of this formula, and does it hold for any metric? . . . 24
Slide 30 — What each term means . . . 24
Question — isn't the left side (true value − prediction)²? . . . 24
Question — but y is still determined by the formula we know? . . . 24
Question — is f̂ the 300 models? . . . 24
Question — why is bias squared and variance not? . . . 24
Question — "the noise is something we add": what about real data? . . . 24
Slide 31 — The straight line (degree 1) at 30 °C, in three numbers . . . 25
Question — how exactly is the variance computed? . . . 25
Question — is 30 °C typical for the straight line? . . . 25
Worked example — the decomposition on five models . . . 25
Slide 32 — Degree 1 at 220 temperatures, −4 to 37 °C, then averaged (optional) . . . 25
Question — why a grid of 220? . . . 26
Slide 33 — Every degree, on 25 days (optional) . . . 26
Question — why does degree 1 have more variance than degree 2? . . . 26
Question — why does degree 12 also have the worst bias²? Flexible models should have low bias. . . . 26
Slide 34 — The bias–variance trade-off: the total error is a U . . . 26
Question — the curves are not the smooth ones of the textbook. Is something wrong? 27
Question — what is the error at degree 7? . . . 27
Slide 35 — The floor you cannot go below (optional) . . . 27
Question — can I ever see a test error below 484? . . . 27
Slide 36 — The best model depends on how much data you have (optional) . . . 27
Question — why does degree 5 become the best with more data? . . . 27
Slide 37 — Without the truth: two errors you can still measure . . . 28
Question — a low training error means low variance, then? . . . 28
Question — "too high" compared with what? . . . 28
Question — if f is unknown on real data, is the decomposition just a proof with no use? . . . 28
Slide 38 — One sample of 25 days, 5-fold cross-validation . . . 28
Question — why not pick degree 5, which has the lowest training error? . . . 28
Slide 39 — Learning curves: would more data help? . . . 29
Question — why AUC now, after twelve slides of errors? . . . 29
Question — "no penalty"? . . . 29
Slide 40 — Learning curves: left the curves meet low, right a gap that stops closing . . . 29
Question — the curves meet, so it is bias? . . . 29
Question — the right-hand curve rises at the end: isn't it still climbing? . . . 30

Question — is the point at 640 the model I will deploy? . . . 30
Slide 41 — Which fix to buy . . . 30
Question — why is prescribing data for a bias problem the expensive mistake? . . . 30
Question — "remove features" as in dropout? . . . 30
Question — can I diagnose on the test error? . . . 30
Slide 42 — Notebook 2, self-study (optional) . . . 30

Part IV — Leakage that survives cross-validation **31**
Slide 43 — Now: what cross-validation does not protect you from . . . 31
Question — what is the F-test? . . . 31
Question — how can random columns look predictive? . . . 31
Question — where does the training happen? Only cross-validation is named. . . . 31
Slide 44 — The code almost everyone writes . . . 31
Question — so the error is that SelectKBest is outside the pipeline? . . . 31
Slide 45 — 0.931 . . . 32
Question — why don't five fairly consistent folds expose the leak? . . . 32
Slide 46 — Fix it, and look what happens . . . 32
Question — why 20 seeds? . . . 32
Slide 47 — The fix, drawn: the selector sees only training rows . . . 32
Slide 48 — Why it does not return to 0.5 . . . 33
Question — why does each of the three help? . . . 33
Question — is the honest pipeline still leaking, then? . . . 33
Slide 49 — GridSearchCV: try all, keep the best . . . 33
Question — what exactly is best_score_? . . . 33
Question — 25 values of C times 5 columns, 125 fits? . . . 34
Question — is k in the grid the number of folds? . . . 34
Slide 50 — GridSearchCV, drawn: 25 cells, 5 folds each . . . 35
Slide 51 — The best of 25 scores is inflated: 0.800 for a worthless model . . . 35
Question — so taking the best of the 25 gives an inflated result? . . . 35
Worked example — the best of five worthless candidates . . . 36
Slide 52 — Nested cross-validation: slide 10's recipe, five times . . . 36
Question — how does it work, step by step? . . . 36
Question — why is it more honest? . . . 36
Slide 53 — Nested cross-validation: it measures what choosing costs . . . 37
Question — are the inner folds the four outer blocks? . . . 37
Question — is 0.670 the score of the model I ship? . . . 37
Slide 54 — Each piece of discipline costs score: 0.931, 0.800, 0.670, 0.500 . . . 38
Question — why does nested cross-validation come closer to 0.5? . . . 38
Question — then why 0.670 and not 0.500? . . . 38

Part V — The debt, and what to report **39**
Slide 55 — The debt, paid: the honest threshold costs 1,400 EUR more . . . 39
Question — the honest threshold costs only slightly more than the cheating one? . . . 39
Question — what else does the table say? . . . 39
Worked example — three thresholds, six validation drives, six test drives . . . 40
Slide 56 — Three sets, three jobs . . . 40
Question — can I use the test score to decide which model to submit? . . . 40
Slide 57 — A fixed seed buys repeatability, not stability (optional) . . . 40

Question — if seed 3 always gives 1.000, isn't that a reproducible result? 40
Slide 58 — What to fix, and what to write down (optional) 41
Question — random_state for scikit-learn, a seeded generator for my own draws? . . 41
Slide 59 — Notebook 3, live 41
Slide 60 — What to put in a report 41
Question — where does the spread come from, for a cross-validation? 41
Slide 61 — What to take away 41
Question — what do all the problems of the lesson have in common? 42
Slide 62 — Homework 42
Question — is it only about fixing the colleague's code? 42

What this document is. A companion to the slides of lesson 5: for every slide, what it says, the steps it leaves implicit, and the questions it tends to raise, answered under the slide they belong to. It is a second explanation of the same material, in a different voice from the handout, which remains the reference for the derivations. It is **not examinable**: nothing here is needed for the exam that is not already in the handout and the notebooks.

Slide numbers and titles are the ones in the lesson's slide deck, in `Slides/`. Where a section mentions "the notes", it means the speaker notes stored in the slide deck, which PowerPoint shows below each slide. The questions are the ones that came up while the lesson was studied and reviewed. The atoms mentioned here are the short predict-then-run examples in `Consolidation/05_lesson_5_atoms.ipynb`.

Worked examples sit under five slides (8, 16, 31, 51, 55): a handful of invented values each, small enough to work through by hand. Their numbers are not the lesson's data; every one is computed by `commentary_examples.py`, beside this file, which also writes them here.

Slide 1 — Lesson 5: Experimental Methodology

Lessons 3 and 4 built two models and measured them. Lesson 5 asks how far those measurements can be trusted. It is the methodology lesson, and it sits in the middle of the course on purpose: every model from lesson 6 on is evaluated with what is built here.

The lesson has one thread. **A score is a measurement**, and like every measurement it has an error bar and can be biased. The first half is about the error bar: how much a score moves when only the luck of the split changes, and how cross-validation reduces that. The second half is about bias in two senses: the bias of a model that is too simple (bias and variance), and the bias of a reported score that has looked at data it should not have seen (leakage).

Part I — A score is a measurement

Slide 2 — Before we start: lesson 4, concluded

This Friday opens with the end of lesson 4: receiver operating characteristic (ROC) curves, the area under the curve (AUC), the threshold chosen from the costs, and its notebook 3. Lesson 5 then has

about two hours instead of three. Nothing is cut from the material: the slides marked (optional) and the three notebooks are left for self-study, and in class short atoms take the notebooks' place. Exercise 4 is discussed on Friday 13 November, exercise 5 on Friday 20 November.

Slide 3 — We broke a promise on purpose

Lesson 1 gave a rule: never let the test set influence any choice. Lesson 4 broke it at its end, by trying many thresholds on the test drives and keeping the cheapest. It was deliberate: the machinery to do it properly did not exist yet. Today builds that machinery and, on slide 55, pays the debt.

Question — so why did lesson 4 do it at all, if it was forbidden?

To show how the costs of the two errors move the threshold, before validation sets had been taught. Lesson 4 had two thresholds. The first, $t^* = 140 \div 2{,}740 = 0.051$, comes from the costs alone and was fixed before looking at any data; it is legitimate. The second was the cheapest of many thresholds tried on the test set itself; that one is the debt. The debt is also wider than the threshold: every choice, of model, hyperparameters, features or threshold, must be made without the test set.

Slide 4 — Today: how to tell whether it works

The agenda in six steps: how far two honest splits can disagree; cross-validation and what it estimates; bias, variance and the error no model can remove; learning curves; leakage that survives cross-validation; and lesson 4's threshold done properly.

Question — the word "bias" seems to mean two things today. Does it?

Yes, and it is worth separating them now. On slide 5 and in the leakage slides, **bias** is the bias of an *estimate*: a measured score that is systematically too high or too low. On slides 26 to 41 it is the bias of a *model*: a model too simple to follow the truth, wrong in the same way on every training set. A natural slip is to merge them, "cross-validation removes bias". It removes neither: it makes an estimate more precise, which is a third thing.

Slide 5 — A test score is a measurement

You would not report a poll of forty people to three decimals: with forty answers and a proportion near one half, the standard error is $\sqrt{0.25/40} = 0.079$, so the second decimal is already uncertain. A test score is the same kind of object. Holding data out makes it **unbiased**: right on average, over all the test sets you might have drawn. It does nothing to make it **precise**: close to the truth on the one split you did draw.

Question — unbiased estimate of what, exactly?

Of the **expected risk** R of lesson 1: the error over the whole distribution of drives the model will ever meet. The test score itself *is* an empirical risk, computed on the test rows. Averaged over

all the test sets one might draw, it equals R. That holds because the test rows played no part in choosing the model. A predictable slip is "the test score is an unbiased estimator of the empirical risk": it is the empirical risk; what it estimates is the expected one.

Question — what does unbiased mean, if two correct splits give different scores?

It means that over many repetitions the scores are centred on the true value; it says nothing about how far any single one lands. The next two slides measure how far: on this fleet, a standard deviation of 0.024 from split to split.

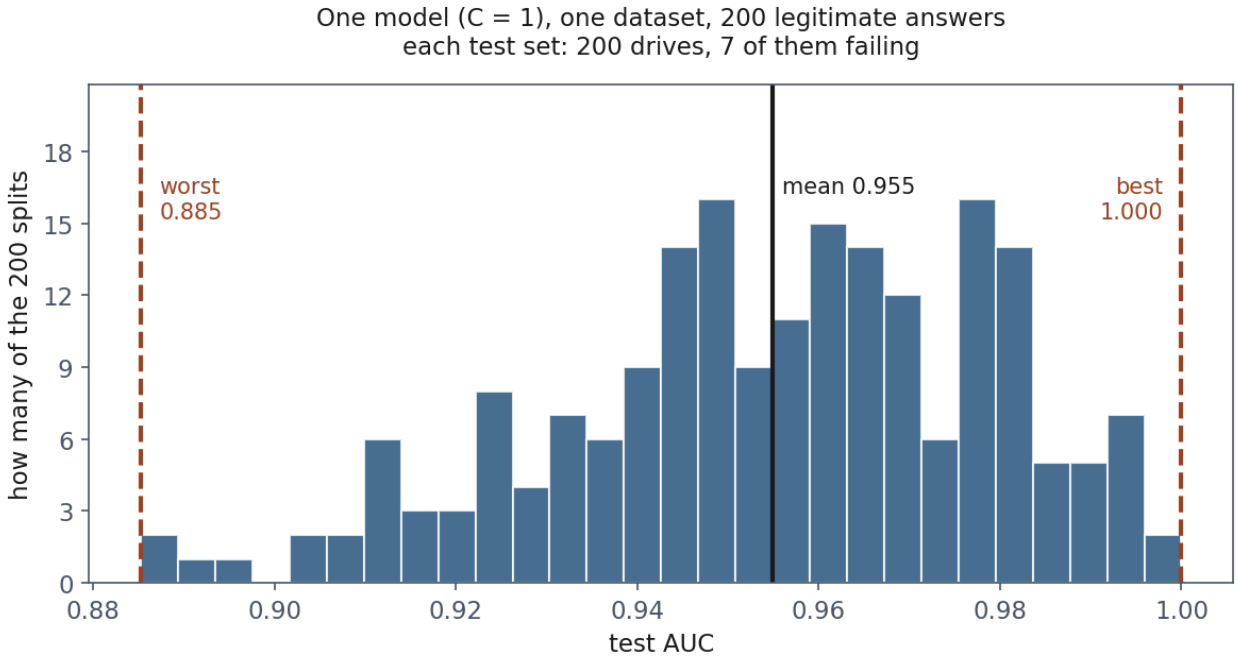
Slide 6 — The fleet, cut down

The data are lesson 4's drives, cut to **800**, of which **29** fail (3.6%). The model is lesson 4's, unchanged: scaling, then logistic regression, scored by the AUC on a stratified 75/25 split. The "In code" line reads in the order the code runs: `train_test_split`, then the pipeline fitted on the training part, then `roc_auc_score` on the test part.

Question — why cut the fleet to 800? Is the demonstration rigged?

No, and the slide's second bullet is the reason. Real projects often start with a few hundred labelled rows, a few dozen of them positive: a pilot study, one year of failures in a small fleet. Lesson 4's 8,000 drives with 306 failures was the comfortable case: a 25% test set held $306 \times 0.25 = 76.5$, so 76 failures. At 800 drives the test set holds 7, and that is the situation this lesson is about. Same data, same model, same code; only the sample size changed.

Slide 7 — 200 honest splits, one dataset



Two hundred legitimate experiments on the same 800 drives. Look at the two dashed lines: the worst split scores 0.885, the best a perfect 1.000.

Two hundred splits, each done correctly: nothing fitted before the split, the test set never inspected. On each, the model is refitted on that split's training part, so these are 200 slightly different models from one procedure. Worst 0.885, best 1.000, mean 0.955, standard deviation 0.024. Each bar is a bin about 0.004 of AUC wide, $(1.000 - 0.885) \div 28$; its height is how many splits landed in it.

Question — which of the 200 values is the true score?

None of them exactly. Each is a legitimate measurement from a slightly different model, on a different 200 drives; their spread is the point of the slide.

Question — why does the notes call the right-hand tail the dangerous one?

Because of what people do with a result. A disappointing score makes you frown and keep working. A delightful one makes you stop and write it in the report. Ten of the 200 splits scored above 0.99; the one at 1.000 reports a *perfect* classifier, every failing drive ranked above every healthy one, for a model whose average is 0.955.

Slide 8 — Why do the 200 scores spread so widely?

Because each test set holds only **7 failures**: 25% of 29 is 7.25. The AUC is a fraction of pairs: each pair is one failing and one healthy drive, $7 \times 193 = 1{,}351$ pairs, and the AUC is the share of them ranked the right way round. With so few failing drives the pairs come in seven bundles of 193, and one drive ranked a little lower moves a whole bundle at once.

Question — why does one failing drive moving change the AUC by 0.015?

Every pair weighs $1 \div 1{,}351$. If one failing drive slips below twenty more healthy ones, twenty pairs turn from right to wrong: the AUC loses $20 \div 1{,}351 = 0.015$, for example from $1{,}351 \div 1{,}351 = 1.000$ to $1{,}331 \div 1{,}351 = 0.985$. The twenty is an illustration, about a tenth of the 193 healthy drives, not a measured value. It is over half the 0.024 standard deviation of slide 7.

Question — "each test set holds 7% failures", right?

Seven failures, not seven per cent: $7 \div 200 = 3.5\%$, the fleet's 3.6% within rounding. And the AUC is not "how many times a failing drive outscores a healthy one", but that count divided by 1,351: a fraction between 0 and 1. Both are slips the slide's wording invites.

The question to carry away: **how many positive examples are in the test set?** Under about fifty, treat every metric as provisional.

Worked example — the AUC as a count of pairs

A test set of seven drives, two failing and five healthy, with the model's predicted probabilities. Healthy: 0.80, 0.60, 0.40, 0.30, 0.20. The pairs are $2 \times 5 = 10$, and a pair is ranked right when the failing drive scores higher.

failing drive's probability	healthy drives it outscores	pairs ranked right
0.90	0.80, 0.60, 0.40, 0.30, 0.20	5 of 5
0.70	0.60, 0.40, 0.30, 0.20	4 of 5

AUC = pairs ranked right ÷ pairs = 9 ÷ 10 = 0.90. Now let the second failing drive score 0.50 instead of 0.70: it drops below one more healthy drive, 1 pair turns wrong, and the AUC becomes 8 ÷ 10 = 0.80. With so few pairs, one drive moving a little moves the AUC by a tenth: slide 8's 1,351 pairs are many more, and still only seven drives carry them.

Slide 9 — Parameters and hyperparameters

Parameters are what the fit estimates: the coefficients w and the intercept b. **Hyperparameters** are what it does not: the penalty λ of lesson 3, the number of neighbours of lesson 6, the depth of a tree in lesson 7, and the threshold of lesson 4. The third bullet is the one that matters most: training error cannot choose a hyperparameter, because it always prefers the most flexible answer, $\lambda = 0$. So hyperparameters are chosen by **comparing** models on data the fit did not see.

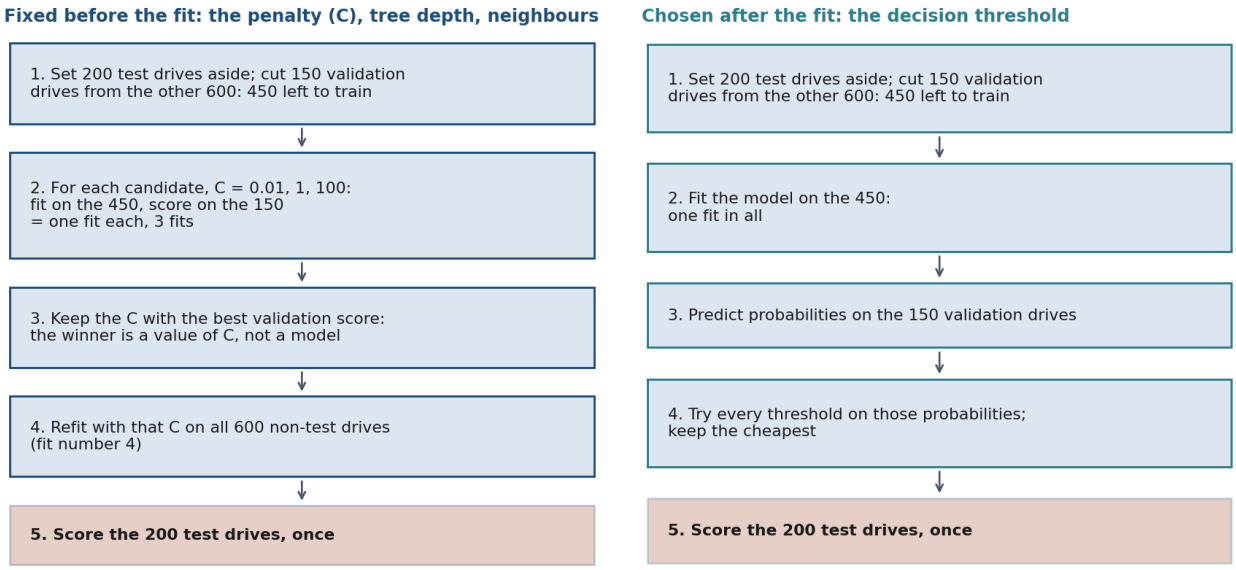
Question — so hyperparameters are the ones chosen before the fit?

That is the tempting shorthand, and the slide warns against it. λ is fixed before the fit, because it sits inside the cost being minimised. The threshold is chosen *after* it, on probabilities the fit has already produced. What they share is not their timing but that neither is estimated the way a coefficient is.

Question — why is scikit-learn's C "the role of 1/ " and not simply 1/ ?

Smaller C means a stronger penalty, the opposite direction to λ. scikit-learn multiplies the data term by C instead of multiplying the penalty by λ; up to a factor of m the two are the same thing, which is why the slide says "plays the role of". The practical consequence is on slide 11: C = 0.01 is the most heavily penalised of the three models, not the least.

Slide 10 — Choosing a hyperparameter on a validation set: two recipes



Both recipes on slide 55's split of the 800 drives: 200 to test, 150 to validate, 450 to train. Step 3 on the left is the one to read twice.

This is where the third set enters. Training error cannot choose, and the test set must not, so part of the non-test data is held out for choosing: the **validation set**. Training fits, validation chooses, test reports once. The two recipes differ in where the hyperparameter acts.

- **Left, fixed before the fit** (C, depth, neighbours). Each candidate needs its own fit: three values of C are three fits on the 450, three AUCs on the 150. The best AUC picks a *value* of C. That value is refitted on all 600 non-test drives (fit number 4), and the test set is scored once.
- **Right, chosen after the fit** (the threshold). One fit on the 450 serves every candidate: predict probabilities on the 150, try every threshold, keep the cheapest, then score the test set once.

Question — what is the validation set for, and how is it used?

For **choosing**; the test set is for **measuring**, once. If you chose on the test set, its score would stop being a measurement: lesson 4's broken promise. So the data are cut three ways. For a hyperparameter inside the cost: one fit per candidate on the training part, one score each on the validation part, keep the best, refit on training plus validation, test once. For the threshold: one fit, probabilities on the validation part, sweep, test once. Cross-validation, on slide 13, is the same idea with the validation part rotated.

Question — in step 3, which model wins?

None: a **number** wins. The three fitted models only served to compare the three values of C and are thrown away. The refit in step 4 is a new model, fitted on more data than any candidate saw. "Identify the winning model, then refit it" is a natural slip; the winner is the value 0.01, or 1, or 100, not a fitted object. Atom 1 runs this recipe and prints it.

Question — why does the left recipe need one fit per candidate and the right one a single fit?

Changing C changes the cost being minimised, so a new optimisation has to run. Changing the threshold only changes how probabilities already computed become decisions.

Slide 11 — On each of slide 7's 200 splits, three models compete

The same 200 splits as slide 7. On each, three logistic regressions differing only in C are fitted on the training part and scored on the test part, and the one with the highest AUC is declared the winner. C = 0.01 wins outright 89 times, C = 1 42 times, C = 100 38 times; $89 + 42 + 38 = 169$, and the other 31 splits are ties.

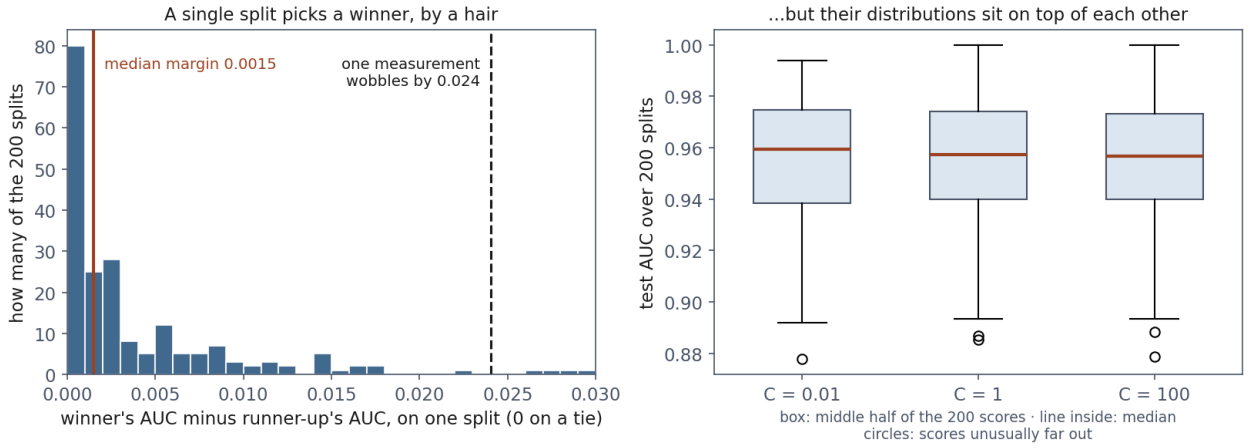
Question — what does the third column count?

On how many of the 200 splits that model had the highest test AUC of the three, which is exactly what you would do in real life with one split: declare the best one the winner. The ties are real: the AUC moves in steps of $1 \div 1{,}351$, and C = 1 and C = 100, both barely penalised, often rank the 200 test drives in exactly the same order; 29 of the 31 ties are between those two.

Question — C = 0.01 wins 89 times against 42: isn't it the better model?

Not on this evidence. Read the second column: the three mean AUCs are 0.9549, 0.9549 and 0.9544. The counts say the models are *different*: C = 0.01 ranks the drives a little differently from the two near-twins, so it wins or loses alone while they split their wins. They say nothing about which is *better*; for that, the means, and the means are equal. Run this once, as real life allows, and you would report one of them as best, with a number to support it: noise wearing the clothes of a result.

Slide 12 — The same three models: who wins, and how they score



Left, the winner's margin on each split; right, each model's 200 scores.

The left panel answers "by how much does the winner win?". On each split the three AUCs are ranked; the margin is the winner's AUC minus the runner-up's, 0 on a tie. The median margin is

0.0015. The dashed line is 0.024, how far a single measurement wobbles from split to split: sixteen times wider. The right panel shows each model's 200 test AUCs as a box plot: the box holds the middle half of the scores, the line inside it is the median, the whiskers reach the rest, the circles are the few scores unusually far out. The three boxes sit at the same height.

Question — what does "winner's AUC minus runner-up's AUC" mean?

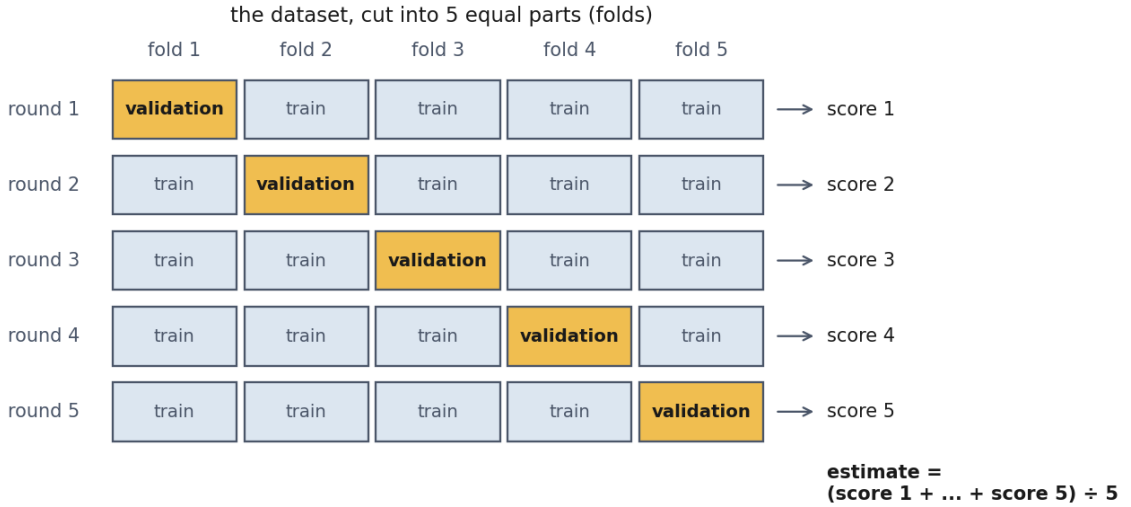
On one split, line the three AUCs up from highest to lowest. The first is the winner, the second the runner-up; the margin is the first minus the second, and the third does not count. On split seed 1, the first without a tie: C = 0.01 scores 0.9756, C = 100 0.9667, C = 1 0.9660. The winner is C = 0.01 and the margin is $0.9756 - 0.9667 = 0.0089$. The histogram is that subtraction done on all 200 splits.

Question — so the message is that there always seems to be a winner, but the models are equivalent?

Yes, and each panel says half of it. The left one says the margin is far too small to trust the choice: a winner is declared on 169 of 200 splits, but by a hair. The right one says the models are in fact equivalent. Together: a single experiment always names a winner, and the winner is decided by which drives landed in the test set.

Part II — Cross-validation, and what it estimates

Slide 13 — k-fold cross-validation: the validation set, rotated



Each row is one round, each column one fold. In each round the gold fold is held out.

If one measurement is noisy, take several and average them; the obstacle is that several test sets cost rows. Cross-validation rotates instead. The 800 drives are cut into 5 folds of 160; in each round a model is fitted on four folds, 640 drives, and scored on the fifth, 160. Five rounds, five scores, and the estimate is their mean. Every drive is predicted exactly once, by a model that never saw it: that is what makes it legitimate rather than testing on training data with extra steps.

Question — does k-fold run on the validation set?

On neither the validation set nor the test set: on all the data that is not the test set. It *replaces* the fixed validation set, whose role the folds take in turn. It has two uses. To **measure** a procedure, with nothing being chosen, it runs on all 800 drives and needs no test set. To **choose**, it runs inside slide 10's recipe: the test set is set aside first, and for each candidate the cross-validation on the rest gives a mean; three values of C cost $3 \times 5 = 15$ fits, the best value is refitted on all non-test data (fit 16), and the test set is scored once. That loop is `GridSearchCV`, slide 49.

Question — one model per fold, or one per round?

One per **round**. A fold is a part of the data; a round is one fit and one score. With k-fold there are as many rounds as folds, which is why "one model per fold" is said, but it later makes "every fold trains on 80%" sound as if a fold were a model. scikit-learn calls a round a split: `n_splits`.

Question — if the block is called validation, why does scikit-learn call it test?

In scikit-learn's names (`test_index`, `mean_test_score`) "test" means only "not used to fit this round's model", never the test set reported once. The lesson keeps lesson 1's words: training fits, validation chooses, test reports once.

Slide 14 — What it actually estimates

The number: at seed 0 the five round AUCs are 0.929, 0.958, 0.984, 0.939 and 0.947, and their mean is $4.757 \div 5 = 0.9514$. Four statements follow. It scores a **procedure**, the recipe as a whole, not one model. It is slightly **pessimistic**, because each round's model saw 80% of the data and the one you ship will see all of it. With **nothing chosen**, this mean is the measurement; with **something chosen**, the test set goes aside first. And in code, `cross_val_score` returns scores, not a model.

Question — so to measure a model I need no test set, and to tune one I do?

Yes, with two precisions. "No test set" holds only if *nothing at all* was chosen on those data: no other models, features or hyperparameters compared. And when you tune, the winner is a value of the hyperparameter, not a model; with cross-validation there is no separate validation set, so the refit is on all the non-test data.

Question — what model do I ship, then?

The recipe again, fitted once on all 800 drives. Keep two words apart: the **recipe** is the instructions (scale, then logistic regression with C = 1); a **model** is the recipe fitted to data, six coefficients and an intercept. Measuring and shipping are two phases:

```
model = make_pipeline(StandardScaler(), LogisticRegression(C=1))    # the recipe
scores = cross_val_score(model, X, y, scoring="roc_auc",
                         cv=StratifiedKFold(5, shuffle=True, random_state=0))
scores.mean()        # 0.9514: what the recipe is worth; five models made and discarded
model.fit(X, y)      # the model you ship, on all 800 drives
```

Five test cakes, each with four fifths of the ingredients, judge a recipe; the cake you bring to the wedding is a sixth, baked with all of them.

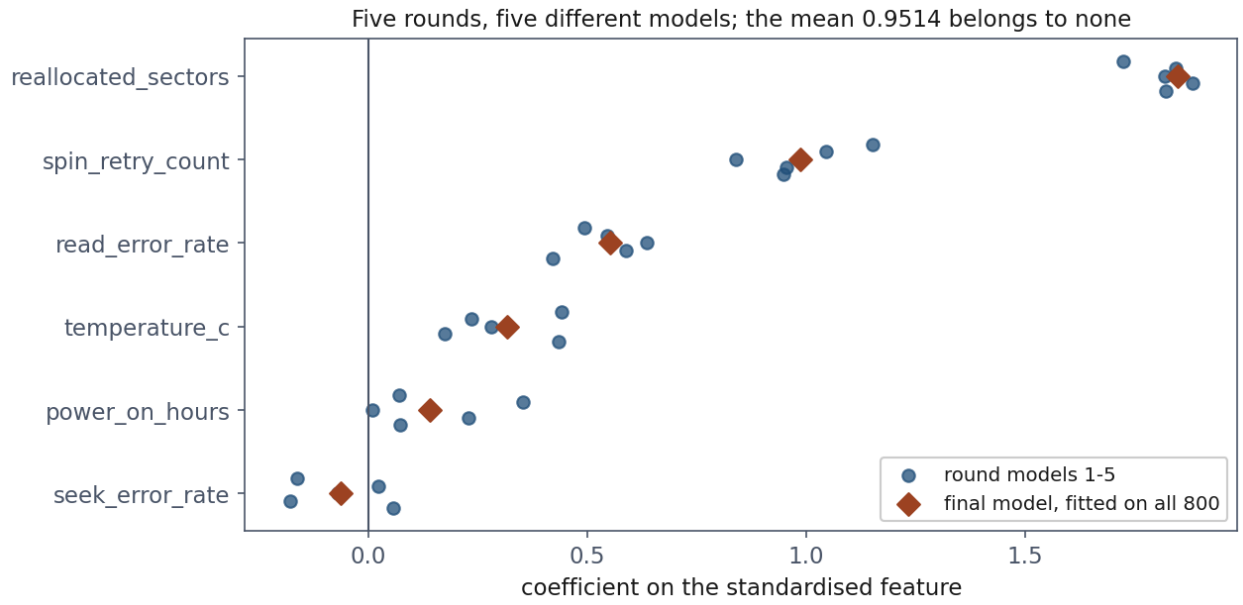
Question — and if the score is not good enough, I change the recipe and cross-validate again?

That is a choice made on the score, and it costs. Try several recipes on the same folds, keep the best, and its score is inflated by the luck of the winner (slide 51 measures how much). If more than one recipe will be tried, set the test set aside before the first. Cross-validation alone is honest for one recipe decided in advance and measured once.

Question — why "slightly pessimistic"?

Each round's model trains on 640 drives and the shipped one on 800; with more data a model is usually a little better, so the estimate errs low, in the safe direction. The gap shrinks as k grows.

Slide 15 — Five rounds, five different models; you ship a sixth, fitted on all 800



One row per feature. Blue dots: the five round models; red diamond: the model fitted on all 800.

The evidence for slide 14's first statement. The five rounds give five sets of coefficients: `power_on_hours` runs from 0.010 to 0.353, and `seek_error_rate` changes sign, from −0.178 to +0.057. At a threshold of 0.5 the five models disagree on 8 of the 800 drives. So 0.9514 is the mean of five scores of five different models, and the model you ship, the diamond, is a sixth that no round ever scored.

Question — why do the coefficients change, with the same algorithm and the same C?

Because the training rows change from round to round, and each round fits its own scaler too. The six rows of the figure are the six features of the drive, standardised so they can be compared; the intercept is not drawn.

Question — is the shipped model one of the five, or their average?

Neither. The diamond sits among the dots, which invites the reading, but it is a new fit of the same recipe on all 800 drives. The five round models only measured the recipe.

Slide 16 — Report the spread, but not as a confidence interval

The question this slide and the next answer: on **another fleet of 800**, how far would 0.9514 move? A 95% confidence interval answers it: the estimate plus or minus 1.96 standard errors, where the standard error is how far the estimate would move if the whole experiment were repeated on new data. The tempting standard error is $s/\sqrt{5}$, with $s = 0.0187$ the standard deviation of the five round scores: $0.0187 \div \sqrt{5} = 0.0084$, and $1.96 \times 0.0084 = 0.016$. The slide says it is **too narrow**, for two reasons, and gives the rule: report the mean and s, call s a spread, treat small differences as unresolved.

Question — why can I not divide by √5?

The formula assumes five independent measurements. These five share their training data: any two training sets have three of their four folds in common. And s is computed from only five numbers, so it is a rough estimate of the spread, too small more often than not. On this fleet the second reason does most of the work.

Question — the models train on overlapping data, so the scores are correlated?

The training sets overlap, but each score is measured on its own fold, disjoint from the others. Over 300 new fleets the correlation between two round scores is only 0.023, because each score is dominated by its own 160 validation drives. The overlap is the general reason, and on other data it can dominate.

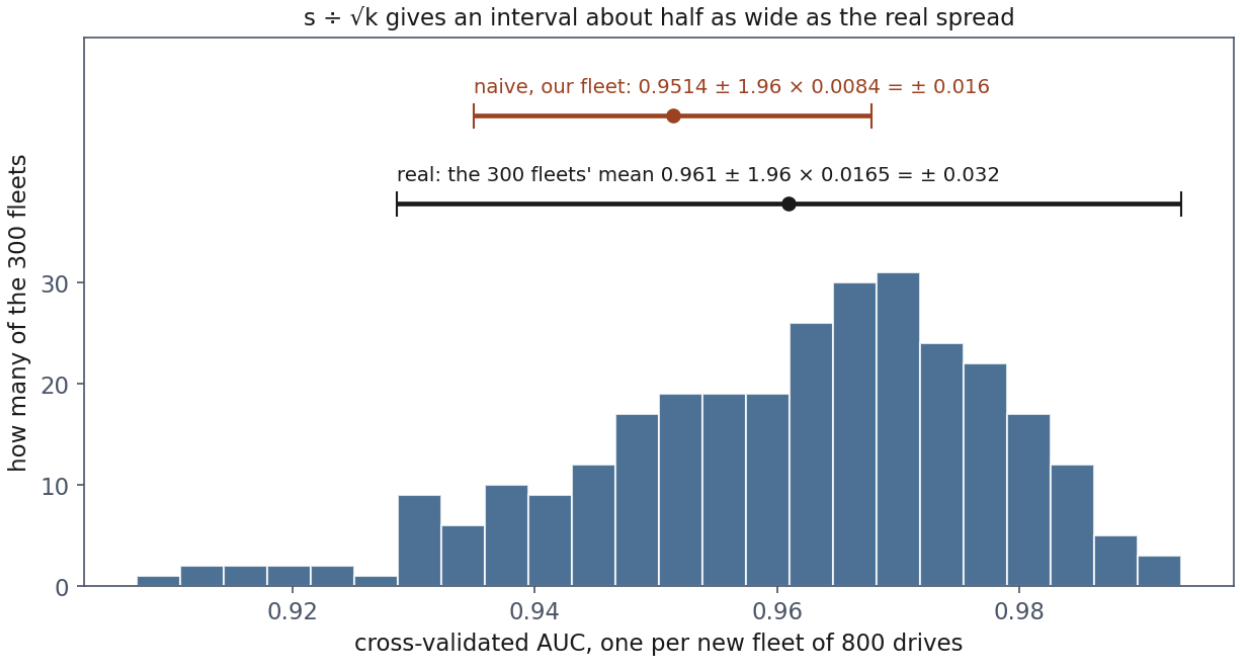
Worked example — five fold scores, their spread and the tempting interval

Five invented round AUCs.

round AUC a_i	deviation $a_i - \bar{a}$	squared
0.90	0.90 − 0.90 = +0.00	0.0000
0.94	0.94 − 0.90 = +0.04	0.0016
0.86	0.86 − 0.90 = -0.04	0.0016
0.92	0.92 − 0.90 = +0.02	0.0004
0.88	0.88 − 0.90 = -0.02	0.0004

The mean is $(0.90 + 0.94 + 0.86 + 0.92 + 0.88) \div 5 = 0.90$. The squares add to 0.0040; divided by 5 that is 0.0008, and its square root is the spread $s = 0.0283$. The tempting standard error is $s \div \sqrt{5} = 0.0283 \div 2.236 = 0.0126$, and the tempting interval $\pm 1.96 \times 0.0126 = \pm 0.025$. Report "0.90 ± 0.028, the spread over 5 folds", not "0.90 ± 0.025": on the drives, slide 17 measured the real spread at twice the tempting one.

Slide 17 — 300 new fleets: twice the naive spread; compare the widths



The histogram is 300 cross-validated AUCs, one per new fleet. Compare the two bars' widths, not their centres.

Notebook 1 does what real data never allows: it draws 300 new fleets of 800 drives from the same generator and runs the same 5-fold cross-validation on each. Their standard deviation is 0.0165, so 95% of them lie within $\pm 1.96 \times 0.0165 = \pm 0.032$. The naive interval promised ±0.016: half as wide.

Question — where do the 300 fleets come from?

From the fleet's generator, not from re-splitting our 800 drives. That is the point: the experiment measures how much the cross-validated score moves with a new fleet, which is the question of slide 16.

Question — the two bars are centred in different places. Which is right?

Both: 0.9514 is our fleet's cross-validation, 0.961 the mean over the 300 new fleets. The slide compares widths. Why it matters: two models at 0.951 and 0.935 look separated with ± 0.016, and are not with ± 0.032.

Slide 18 — Stratify when a class is rare

The 29 failures are among all 800 drives, so a fold of 160 should hold $29 \div 5 = 5.8$ of them: 5 or 6, since drives are whole. `StratifiedKFold` deals them 5, 6, 6, 6, 6. Plain `KFold`, at seed 3, deals 7, 1, 7, 5 and 9, and the fold with a single failure scores a perfect 1.000: that one drive happened to rank above all 159 healthy ones.

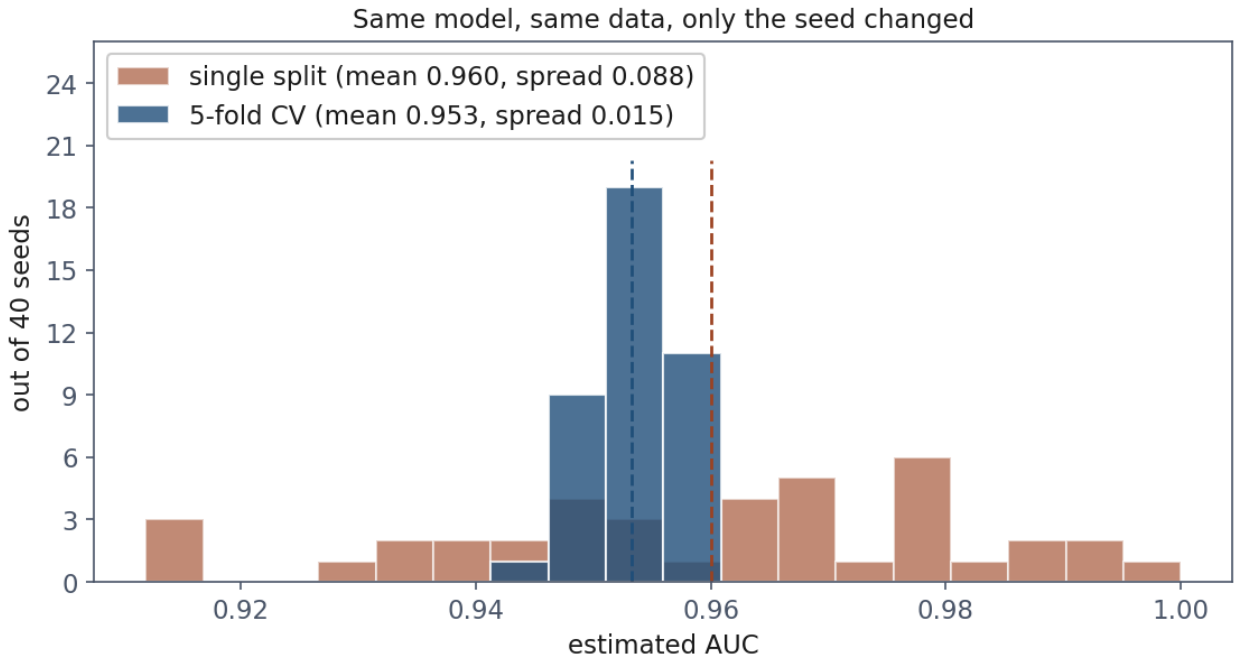
Question — must every fold have the same number of positives?

The same *proportion*, as close as whole drives allow. And stratification applies to every split of the data, not only to the folds: `train_test_split(..., stratify=y)` is why every test set of slide 7 held exactly 7 failures.

Question — does stratification make the scores less spread?

Not on every seed; here the stratified scores spread a little more. What it promises is that no score rests on one drive. It costs nothing: one word in the call.

Slide 19 — Change only the seed, 40 times: how far does the score move? (optional)



The same 40 seeds, single split against 5-fold cross-validation, on the same bins.

A single 75/25 split, 40 seeds: from 0.912 to 1.000, standard deviation 0.0221. Five-fold cross-validation, the same 40 seeds: from 0.944 to 0.959, standard deviation 0.0032, about seven times smaller ($0.0221 \div 0.0032 = 6.9$). The means are close, 0.960 and 0.953: cross-validation is not more optimistic or pessimistic here, it is less arbitrary.

Question — why does the seed matter so much less for cross-validation?

For a single split the seed picks *which* 200 drives are tested, so which 7 of the 29 failures count. For cross-validation it only changes how the 800 are cut into folds: every drive is still predicted once, and all 29 failures count.

Question — so cross-validation is accurate to ± 0.003?

No. 0.0032 is how far it moves when only the fold cut changes on the *same* 800 drives. On new data it moves by 0.0165, slide 17. Cross-validation is less arbitrary, not that precise.

Slide 20 — How many folds? (optional)

Five or ten, in practice. A larger k makes each model train on more data, 80% at $k = 5$ and 90% at $k = 10$, so the estimate is less pessimistic; it also costs more fits, and the training sets overlap more. At $k = m$ every row is a validation fold on its own: leave-one-out.

Question — why are the scores more correlated with a larger k?

Not because the validation fold is smaller, which is the natural guess, but because the training sets overlap more: any two share 75% of their rows at $k = 5$ and about 90% at $k = 10$. Models trained on nearly the same data are nearly the same model.

Question — what is leave-one-out for?

It earns its place with **very few rows and a metric computed row by row**, such as the mean squared error or accuracy. Each model trains on $m - 1$ rows, so the estimate is nearly unbiased, and there is only one way to cut, so no seed: with 30 patients, losing a fifth of them per fold costs a lot, losing one costs almost nothing. The price is m fits and a high-variance estimate, since the m models are nearly the same and each score rests on one row. With the AUC it cannot work at all: a fold of one row holds no failing-healthy pair, so `cross_val_score` returns `nan` for every round.

Question — is k a hyperparameter?

No. It is a setting of the *measurement*, not of the model: the model you ship is refitted on all the data whatever k was. Fix it beforehand and never choose it by the score it gives. If compute is spare, repeated 5-fold buys more than moving to 10 folds: over 40 seeds the spread of the estimate falls from 0.0033 to 0.0009 with ten repetitions, while 10 folds give 0.0036.

Slide 21 — When the rows are not independent

Suppose the table held ten readings per drive instead of one. A random 5-fold split puts about 8 of a drive's 10 readings in training and 2 in validation, and those 2 are near-copies of the 8, with the same label. The model can recognise the drive instead of learning anything about failure. A 1-nearest neighbour, which copies the label of the closest training row, scores 1.000 with rows split at random and 0.620 with each drive's readings kept together.

Question — how can the model recognise a drive it was never told the name of?

A drive's readings have nearly the same values and the same label. A flexible model finds the training reading closest to a validation reading, which is the same drive's, and copies its label: a lookup, not a pattern. It is a form of leakage, since the validation row is not independent of the training rows.

Question — does every model fall for it?

Flexible ones do. In the handout's simulation the nearest neighbour drops from 1.000 to 0.620 and a random forest from 0.999 to 0.896, while logistic regression, with six coefficients, cannot memorise 800 drives: 0.960 against 0.950. "8 of 10" is an average; one reading on the training side is already enough.

Slide 22 — Split by the thing you must generalise to

The cure: **GroupKFold**, which keeps all of a drive's rows in one fold. It needs to know which rows belong together, and that is what `drive_id` is: a column with one value per row, the serial number

of the drive the reading came from.

row	reallocated sectors	…	failed (y)	`drive_id`
0	1.39	…	0	0
1	1.43	…	0	0
… rows 2 to 9			0	0
10	1.60	…	1	1
…				…
7,999	1.39	…	0	799

It comes from the data, as a medical record carries the patient's id. It is **not a feature**, because a model that saw it could recognise the drive again; it is passed apart, as `groups=drive_id`. Each round then trains on 640 drives, 6,400 readings, and validates on 160 others, 1,600 readings. Atom 8 prints these rows and runs the call.

Question — how do I pass the groups, and what about the test set?

`cross_val_score(model, X, y, groups=drive_id, cv=GroupKFold(5))` for the folds. The final test split must be grouped too: `train_test_split` takes no groups, `GroupShuffleSplit` does. With a rare class, `StratifiedGroupKFold` groups and stratifies at once.

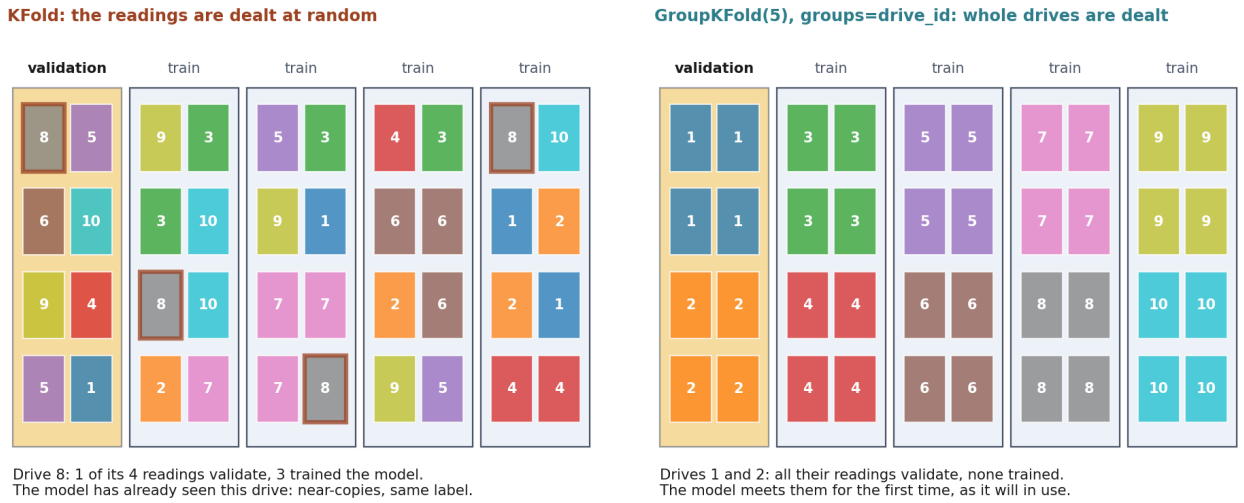
Question — do the groups have to be the same size?

No: one drive with ten readings among drives with one each is split correctly. What unequal groups change is weight: that drive counts ten times in training and in the score, so aggregate to one row per drive, or weight its readings 1/10, depending on whether the model will judge readings or drives.

Question — what about data ordered in time?

If the model must predict forward, validation must be forward too, and scikit-learn has `TimeSeriesSplit` for it. This course does not treat time series, so the slide leaves it out.

Slide 23 — GroupKFold, drawn: each drive in one fold



Drawn small: 10 drives, 4 readings each, one square per reading, numbered by drive. On the lesson's data, 800 drives and 10 readings each:
1-nearest neighbour scores 1.000 split at random and 0.620 split by drive.

Left, readings dealt at random: drive 8 has one reading in the validation fold and three in training. Right, whole drives dealt: drives 1 and 2 validate and none of their readings trained the model.

The previous slide in a picture, drawn small so every square can be read: ten drives, four readings each, one square per reading, numbered by drive. `KFold` deals readings one at a time, so a drive ends up on both sides of the split, and the model is graded on near-copies of rows it trained on, with the same label. `GroupKFold` deals whole drives, so every validating drive is one the model has never seen, which is the situation it will meet in use. On the lesson's data, 800 drives with ten readings each, a 1-nearest neighbour model scores 1.000 with readings dealt at random and 0.620 with drives dealt whole (slide 21). The second number is the one to report.

Slide 24 — Notebook 1, live

The table maps notebook 1's sections to the slides that introduced them and to the number to come back with: seed 3 gives 1.000; 200 splits run from 0.885 to 1.000; three values of C win 89, 42 and 38 times; cross-validation gives 0.9514, and the real spread over new fleets is 0.0165 against the naive 0.0084; stratification turns a fold of one failure and AUC 1.000 into five folds of 5 or 6. In class atoms 2 and 3 run in its place, and the notebook is self-study.

Slide 25 — Break

Ten minutes. The second half opens with a practical question.

Part III — What the error is made of

Slide 26 — Error too high: more data, or another model?

The second hour opens with a decision. Daily energy use against the outdoor temperature, as in lesson 3; a straight line fitted on 25 days has a 5-fold cross-validated mean squared error (MSE) of 6,064.3 kWh², a typical miss of √6,064.3 78 kWh a day. Too high. Two cures are on offer, more days or another model, and each cures a different kind of error: buy the wrong one and the month changes nothing. To choose, find where the error comes from, first in a lab where the truth is known, then on real data. Slide 41 gives the answer.

Question — is 6,064.3 measured on a validation set?

On no fixed set: it is a 5-fold cross-validation on the 25 days. Each round fits the line on 20 days and scores it on the other 5, and 6,064.3 is the mean of the five validation MSEs. The test set is not touched while we diagnose.

Question — is it lesson 3's dataset?

The same setting and the same noise, 22 kWh, but a new curve: lesson 3 used a plain parabola, this one carries a seasonal term, so no polynomial fits it exactly. And the 25 days are not consecutive: they are scattered over the year, each with its own daily mean temperature.

Question — why go through a lab at all, if cross-validation does the diagnosis?

Cross-validation gives two numbers; the decomposition of slide 29 says what they mean. The formula is the *why*, cross-validation the *how*. A doctor reads symptoms in the clinic, but learnt what they mean in physiology, where the mechanism is known; the lab is our physiology.

Slide 27 — A lab where we know the truth

The truth is written down: $f(t) = 240 + 0.55\,(t - 18)^2 + 12\sin(t/4)$ kWh, with t the day's mean outdoor temperature, plus noise of standard deviation 22 kWh. The 18 is the balance temperature of degree days: below it a building heats, above it cools. From this curve 300 new training sets of 25 days are drawn, and polynomials of degree 1 to 12 are fitted to each.

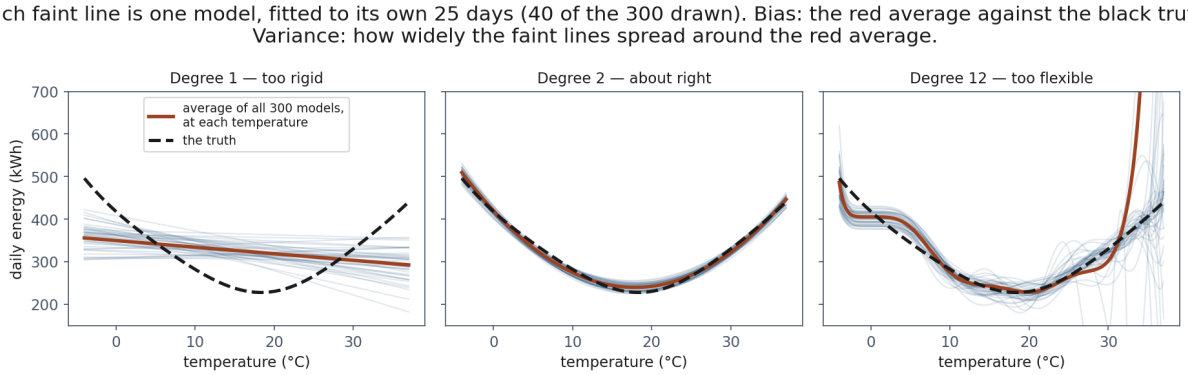
Question — so for degree 3 there are 300 models?

Yes: 300 training sets, one model each. Each set is drawn **afresh** from the curve, with new temperatures and new noise. They are not splits of slide 26's 25 days, which are only the first of the 300. Splits of one sample would share most of their data, give models too alike, and make the variance come out too small.

Question — what are the 300 models for, if in real life I have one?

To see how a model changes when its data change. Their **average** compared with the truth is the bias; how widely they **spread** around that average is the variance. Both are measured at 220 test temperatures evenly spaced from −4 to 37 °C.

Slide 28 — Three hundred parallel universes: one model each



Forty of the 300 models drawn per panel; the red curve is the average of all 300, at each temperature.

Each faint line is one model, fitted to its own 25 days. Left, degree 1: every line misses the curvature the same way, and the red average sits away from the black truth: that gap is bias. Middle, degree 2: the average tracks the truth and the lines stay close. Right, degree 12: the lines fly off at the edges, where all the training points lie on one side.

Question — how is the red curve obtained?

Temperature by temperature: at each of the 220 test temperatures, the 300 models give 300 predictions, and their mean is the red curve's height there. On the left panel at 30 °C the red curve is at 303.6 and the black at 330.5.

Question — degree 1's lines fan out: isn't that bias?

No, the fan is **variance**; bias is the gap between the red average and the truth. Bias is about where the average sits, variance about how far the lines sit from it. A straight line fans out because each universe draws its own temperatures, and a line that cannot follow the curve pivots according to where they fell.

Question — at degree 12 the red curve shoots up at the right edge, though most lines do not. Why?

An average is sensitive to extremes: a few models that explode at the edge drag it up. The enormous variance of degree 12 contaminates its average too.

Slide 29 — The bias–variance decomposition: expected error = bias² + variance + noise

The left side is slide 26's question made precise: the squared error of a model on a new day, averaged. The right side splits it into three parts, two that belong to the model and one to the data. It is the **bias–variance decomposition** of the expected squared error, and it is an identity, not an approximation: the two sides are equal exactly. The handout derives it in five lines.

Question — what is the name of this formula, and does it hold for any metric?

The bias–variance decomposition; its consequence, the U of slide 34, is the bias–variance trade-off. It holds for squared error. For accuracy or the AUC there is no split this clean. History: Geman, Bienenstock and Doursat brought it into machine learning in 1992, in "Neural networks and the bias/variance dilemma".

Slide 30 — What each term means

At a new day with $x = 30$ °C: y is its energy, the truth plus fresh noise; $\hat{f}(x)$ is **one** model's prediction; the expectation $\mathbb{E}$ averages over two random things, which training set the model came from and the noise of the new day. **Bias** is the average prediction minus the truth, in kWh and with a sign; **variance** is the average of (prediction minus average prediction)², the models' disagreement; **noise** is $\sigma^2 = 22^2 = 484$.

Question — isn't the left side (true value − prediction)²?

It is $(y - \text{prediction})^2$, with y the energy measured on a new day: truth plus fresh noise. That is exactly where σ^2 comes from. Put the true value $f(x)$ inside the square instead and only bias² + variance remain. It is the most natural slip of the slide, because that is how everyone pictures an error.

Question — but y is still determined by the formula we know?

Only its distribution: $y = f(x) + \varepsilon$, with $f(30) = 330.5$ fixed and ε drawn afresh every day. Two days at 30 °C give two different energies around 330.5, which is why even a perfect model, $\hat{f} = f$, misses by 484 on average. On real data it is the other way round: y is the one thing measured, and f and σ are unknown.

Question — is $\bar{f}$ the 300 models?

One model. In real life you train one and predict with it; the 300 are averaged only to *measure* how wrong such a model is. The average model exists only in this lab.

Question — why is bias squared and variance not?

Bias is a signed gap, in kWh: −26.9 kWh and +26.9 kWh cost the same. Expanding the average squared error produces the term (truth − average model)² by itself, in kWh² like the other two, which is what lets the three be added. Variance is already an average of squares.

Question — "the noise is something we add": what about real data?

In the lab we add it, so we know it. In real data it is there anyway, the variability of the days that the temperature does not explain, such as weather and occupancy; you just do not know how large it is.

Slide 31 — The straight line (degree 1) at 30 °C, in three numbers

The whole decomposition at one temperature. The truth $f(30) = 330.5$ kWh. The 300 lines' predictions at 30 °C run from 215.6 to 364.7, and their average is sum $\div$ 300 = 303.6. Bias² $= (303.6 - 330.5)^2 \approx 724$; variance, the sum of (prediction $- 303.6)^2$ divided by 300, ≈ 856; noise 484. The expected error is 724 + 856 + 484 = 2,064 kWh², a typical miss of $\sqrt{2{,}064}$ 45 kWh. Read as a sentence: 724 because the line cannot bend, 856 because it depends on which 25 days it saw, 484 because the new day is itself noisy. Atom 9 computes it.

Question — how exactly is the variance computed?

Square each deviation first, then sum, then divide by 300: $[(300.8 - 303.6)^2 + (317.1 - 303.6)^2 + (304.8 - 303.6)^2 + ...] \div 300$. Summing the deviations first and squaring afterwards gives zero, because the deviations from an average cancel out. The truth does not enter the variance at all.

Question — is 30 °C typical for the straight line?

No, it is a lucky spot: there the line passes close to the curve. Averaged over all temperatures its bias² is 5,250.8, not 724. Slide 32 shows why.

Worked example — the decomposition on five models

Five models, each trained on its own data, asked about the same point, where the truth is $f = 10$. The noise of a new measurement has variance $\sigma^2 = 1$.

prediction $\hat{f}$	for the variance: $(\hat{f} - \bar{f})^2$	for the check: $(f - \hat{f})^2$
8	$(8 - 8)^2 = 0$	$(10 - 8)^2 = 4$
9	$(9 - 8)^2 = 1$	$(10 - 9)^2 = 1$
7	$(7 - 8)^2 = 1$	$(10 - 7)^2 = 9$
10	$(10 - 8)^2 = 4$	$(10 - 10)^2 = 0$
6	$(6 - 8)^2 = 4$	$(10 - 6)^2 = 16$

Average prediction $\bar{f} = (8 + 9 + 7 + 10 + 6) \div 5 = 8$. Bias $= 8 - 10 = -2$, so bias² = 4. Variance = middle column summed $\div$ 5 = 10 $\div$ 5 = 2; the truth does not enter it. Check: the last column, the squared misses from the truth, summed $\div$ 5 = 30 $\div$ 5 = 6, exactly bias² + variance = 4 + 2. Against a new noisy measurement add the noise: expected error = 4 + 2 + 1 = 7.

Slide 32 — Degree 1 at 220 temperatures, −4 to 37 °C, then averaged (optional)

The 30 °C computation repeated at every temperature of a grid of 220, evenly spaced from −4 to 37 °C, for the straight line. The red curve is bias² at each temperature: zero near 5 and 28 °C, where the line crosses the true curve, and large at both ends and in the middle. Its average, the dashed line, is 5,250.8. The blue curve is the variance, lowest in the middle where the days are dense; its average is 723.8. The dotted line is the noise, 484 everywhere. The three averages, 5,250.8 + 723.8 + 484.0 = 6,458.6, make the first row of slide 33's table.

Question — why a grid of 220?

The error depends on the temperature, and the table needs one number per degree. 220 is arbitrary: any fine grid gives nearly the same averages. The grid sits just inside the data's −5 to 38 °C, so the models are not judged where no day can fall.

Slide 33 — Every degree, on 25 days (optional)

The same averages for every degree. The minimum is degree 2, 614.3; degree 3 is close behind, 654.4. The noise column never moves, because it belongs to the data. Degree 1 is a bias problem, degree 12 a variance problem (32 million); in between, bias and variance are both small.

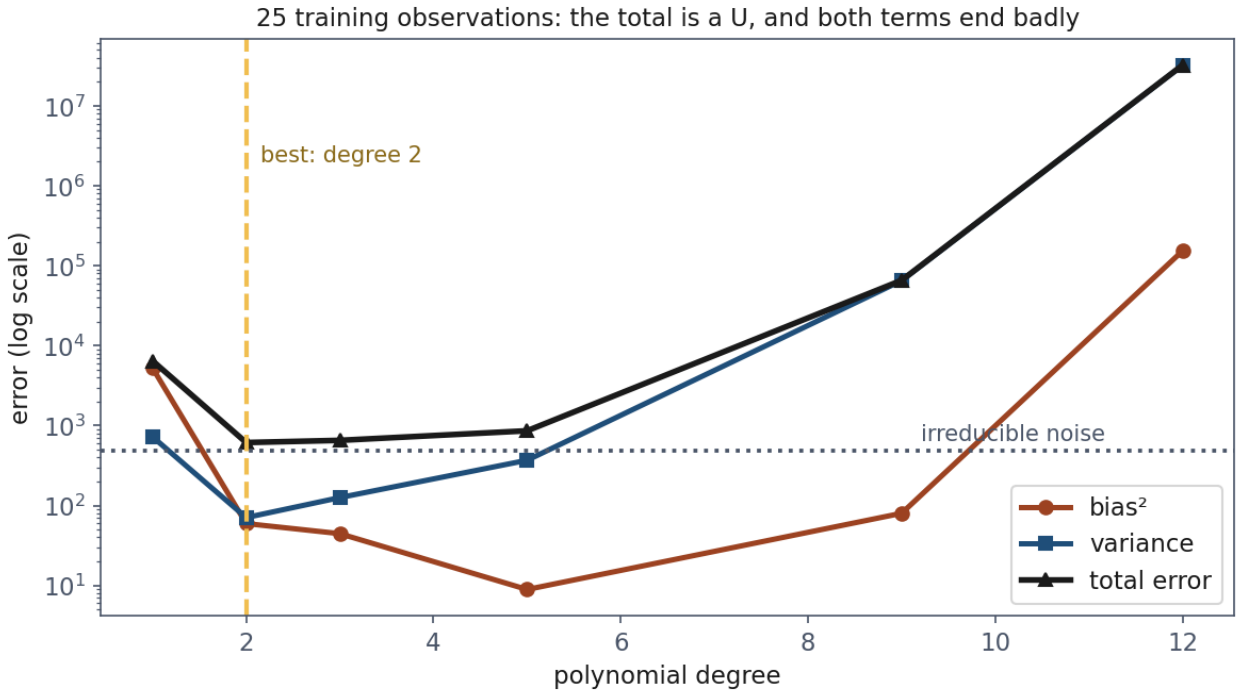
Question — why does degree 1 have more variance than degree 2?

The line pivots: every universe draws its own 25 temperatures, and a line that cannot follow the curve tilts according to where they fell. With the temperatures held fixed, the handout shows, degree 1 becomes the most stable model, as the textbook picture promises.

Question — why does degree 12 also have the worst bias²? Flexible models should have low bias.

What fails is the measurement. A few wild fits dominate both columns, the average of 300 such curves is itself unstable, and another seed gives very different numbers. Degree 12 is unusable, and its exact digits mean nothing.

Slide 34 — The bias–variance trade-off: the total error is a U



Logarithmic vertical axis; the dotted line is the noise floor, 484.

Slide 33's table as a figure: too rigid on the left, too flexible on the right, best at degree 2. The vertical axis is logarithmic, each tick a factor of 10, because on a linear axis degree 12 would flatten everything else onto the baseline.

Question — the curves are not the smooth ones of the textbook. Is something wrong?

No: the textbook picture is a well-specified model on fixed inputs. Measured here, bias^2 falls only as far as degree 5 and then climbs, and the variance starts high at degree 1 before it rises. What survives is what the lesson needs: the total error is a U with its minimum at degree 2.

Question — what is the error at degree 7?

Nobody knows: only degrees 1, 2, 3, 5, 9 and 12 were measured, and the segments between them only join the points.

Slide 35 — The floor you cannot go below (optional)

The noise, 484, is a floor under the expected error on **new** data: no model, algorithm or quantity of data goes below it. So a **test** error well below a known floor means leakage or a mistake in the evaluation, not excellence.

Question — can I ever see a test error below 484?

By luck, on a small test set. The floor bounds the *expected* error; on 5 test days even the true curve scores below half the floor about one time in five. Hence "well below". The training error is another matter: degree 5 trains at 262.4 by fitting the noise. And the floor must be *known*: here because we added the noise, on real data from the instrument's precision or from repeated measurements.

Slide 36 — The best model depends on how much data you have (optional)

The same experiment at 25, 60 and 150 days. The best degree moves from 2 to 5 as the data grow. The straight line barely improves, from 6,458.6 to 5,861.5 with six times the data, because its error is bias. Degree 12 improves by a factor of thousands, because its error was variance.

Question — why does degree 5 become the best with more data?

With more days there is enough data to pin down its extra coefficients, so its variance falls. More data shrinks variance, not bias: that one sentence explains the whole table, and it is the answer slide 26 is waiting for. "Which model is best" is a question about models *and* the quantity of data you have.

Slide 37 — Without the truth: two errors you can still measure

Back to real life: one training set, f and σ unknown. Cross-validation inside the training data, test set untouched, still gives two errors. The **training error**, on the days the model was fitted to, hides its variance, because the model has already bent towards those days. The **validation error**, on the fold it did not see, pays bias, variance and noise. Reading: a training error too high for the task points to bias; a validation error far above the training error points to variance.

Question — a low training error means low variance, then?

No: the training error *hides* the variance, it does not measure it. Degree 12 has a tiny training error and an enormous variance.

Question — "too high" compared with what?

The noise is unknown on real data, so a high training error might be noise alone, which no model removes. The reference has to come from outside: the error the task needs (if forecasts must be within 50 kWh and the line misses by 72 kWh on the days it has *seen*, it cannot do better on new ones); the instrument's known precision; or a more flexible model, whose training error drops if the problem was bias. That last comparison is what separates bias from noise. And the two problems can come together.

Question — if f is unknown on real data, is the decomposition just a proof with no use?

It is a reasonable reading, but the decomposition is what gives the two measurable errors their meaning, and what says which cure each one needs.

Slide 38 — One sample of 25 days, 5-fold cross-validation

Slide 26's 25 days, three degrees, both errors as means over five rounds:

- **Degree 1**: 5,155.7 training, 6,064.3 validation. Both high and *close*: the line is as wrong on days it has seen as on days it has not. Bias.
- **Degree 2**: 312.8 and 450.4, the lowest validation error: the choice.
- **Degree 5**: 262.4 training, below the noise of 484, so it has fitted part of the noise; 615.1 validation, a gap of about 353. Variance.

The numbers differ from the lab's, and must: one sample against an average over 300. The ordering carries over, and a diagnosis needs nothing more.

Question — why not pick degree 5, which has the lowest training error?

It looks like the most accurate, and it is the one chasing the noise. The choice is always made on the validation error. With 150 days degree 5's gap falls from 352.6 to 58.3: more data cured its variance.

Slide 39 — Learning curves: would more data help?

Slide 38 answered "bias or variance?" at one size; slide 26 asked about buying data, and that needs the errors at several sizes. A learning curve repeats the two errors at growing training sizes: `learning_curve(model, X, y, cv=folds, scoring="roc_auc")`. The example switches to slide 6's 800 drives, so the score is the AUC, and the direction flips: higher is better, so a poor model shows as **low** curves. Two models are broken on purpose: one with a single feature, one with 6 features plus 150 noise columns and almost no penalty.

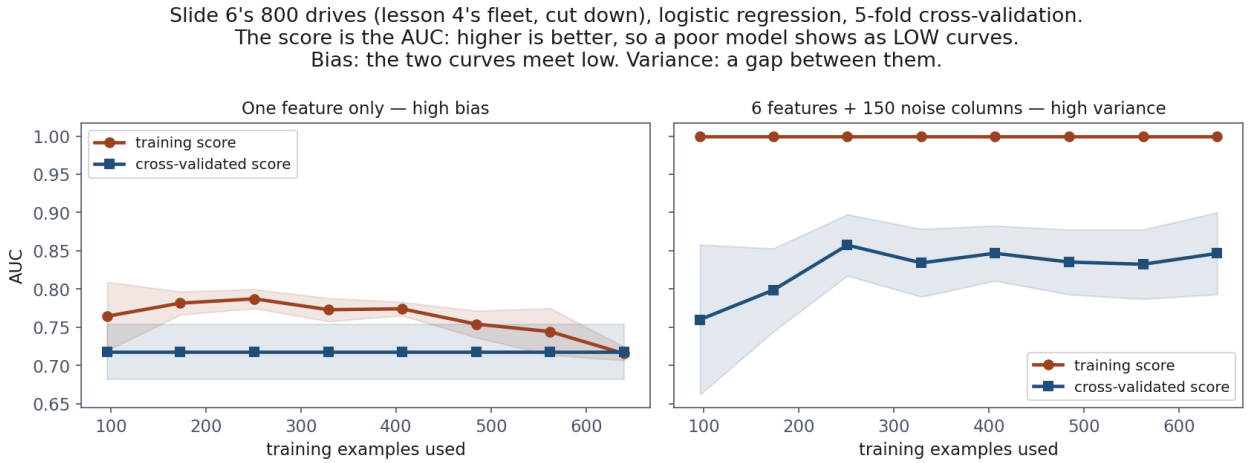
Question — why AUC now, after twelve slides of errors?

Because on the drives the two failures can be shown with real features. Calling the AUC "the error" and reading the curves upside down is the predictable slip.

Question — "no penalty"?

Almost none: C = 1000, a very weak penalty, since C works as $1/\lambda$.

Slide 40 — Learning curves: left the curves meet low, right a gap that stops closing



Left, one feature: the curves meet low. Right, 156 columns: training at 1.000, a gap that stays.

The x axis counts the training drives used, from 96 to all 640 of a round's training part; every point is scored on the same 160 held-out drives and averaged over the five rounds. Left: the curves meet, and they meet **low**, 0.715 against 0.718; the validation curve is exactly flat, because with a single feature logistic regression ranks the drives in that feature's own order whatever coefficient it learns. Bias. Right: training exactly 1.000, the model memorises the drives; validation climbs to 0.857 at 251 examples, then stops and ends at 0.847. A gap of about 0.15 that does not close. Variance.

Question — the curves meet, so it is bias?

They must meet *low*. A good model's curves also approach each other as the data grow, but high, near 1. The word that carries the diagnosis is "low".

Question — the right-hand curve rises at the end: isn't it still climbing?

The last step is +0.014, and across the five rounds it is −0.030, +0.050, +0.006, −0.009 and +0.053: a wiggle inside a band of about 0.04, not a trend. A slope smaller than its band is not a slope.

Question — is the point at 640 the model I will deploy?

No: 640 is the largest training part a round holds; the deployed model is refitted afterwards on all the development data.

Slide 41 — Which fix to buy

Three diagnoses, three cures. Curves meeting at a **poor** score: bias, and more data will not help; change the model, add better features, or regularise less. A wide gap with the validation curve **still rising** by more than its spread: more data is likely to help. A wide gap with the validation curve **flat**: a stronger penalty (smaller C) or fewer features. Slide 26's answer: the line's error is bias, so change the model; degree 2 on the same 25 days brings the validation MSE from 6,064.3 to 450.4. And the test set is scored once, after the fix.

Question — why is prescribing data for a bias problem the expensive mistake?

Collecting data is the slowest and costliest item on the list, and a flat curve says in advance that it will change nothing. The instinct is sound, more data almost never hurts, it just does not help this failure.

Question — "remove features" as in dropout?

No: it means dropping columns, here the 150 noise columns. Dropout is a neural-network technique of lesson 9 with nothing to do with it.

Question — can I diagnose on the test error?

It is the number everyone looks at, so the instinct is natural. But every number in this diagnosis came from cross-validation inside the training data. If the final test score disappoints and you go back and change the model, the test set has become a validation set.

Slide 42 — Notebook 2, self-study (optional)

The map of notebook 2: the 300 universes, the three parts measured (their sum agrees with the measured error to twelve decimal places), the three sample sizes, training against validation without the truth, and the learning curves. On 6 November it is left for self-study; its numbers are the ones on slides 26 to 41.

Part IV — Leakage that survives cross-validation

Slide 43 — Now: what cross-validation does not protect you from

The setting of the last hour. The labels are slide 6's 800 drives, 29 failures; the features are replaced by **2,000 columns of random noise**, independent of the label. There is nothing to find: any model's honest AUC is 0.500. The procedure: keep the 10 columns with the highest F-test, `SelectKBest(f_classif, k=10)`, then cross-validate a logistic regression on them, 5 rounds, training on 640 drives and scoring 160.

Question — what is the F-test?

`f_classif` takes one column at a time and asks: do the failing and the healthy drives have different means in it? It answers with a ratio, how far apart the two groups' means are divided by how widely the values scatter inside each group: an analysis-of-variance (ANOVA) F statistic, which with two classes is the square of the familiar t statistic. A large value says the column seems to separate the classes; `SelectKBest` keeps the ten largest. Note the clash of letters: this k counts columns, not folds.

Question — how can random columns look predictive?

The mean of 29 random values is not exactly 0: it wobbles with standard deviation $1/\sqrt{29} = 0.19$. Across 2,000 columns a few drift far by luck; the ten chosen here have the failing drives' mean shifted by 0.52 to 0.77, between 2.8 and 4.1 times what chance usually allows. They look like signal, and they are the luck of those 29 drives. Atom 11 shows one of them.

Question — where does the training happen? Only cross-validation is named.

Inside it. The data, 800 rows and 2,000 noise columns; the selector looks at all 800 rows once and keeps 10 columns; inside `cross_val_score`, five rounds each train a logistic regression on 640 drives using those 10 columns and score the other 160; the five AUCs are averaged. Cross-validation *is* training and scoring five times.

Slide 44 — The code almost everyone writes

Six lines that read as a sensible sentence: "two thousand features is too many, so let me keep the ten most predictive, then cross-validate a model on those". Every line is defensible in isolation. The slide is bare on purpose: the room is asked to find the bug before the next slide.

Question — so the error is that SelectKBest is outside the pipeline?

Yes, and the precise reason is the sentence to keep. `sel.fit(noise, y)` sees all 800 drives *and their labels*, including the 160 that each round will use for validation. Those drives have already helped choose the columns, chosen because on them, by chance, they separate the classes. The cross-validation repeats only what is in the pipeline, scaling and the model; it cannot assess a step it does not repeat.

Slide 45 — 0.931

Cross-validated AUC **0.931**, the mean of five folds, $(0.928+0.979+0.975+0.943+0.832) \div 5 = 0.931$, on data with no signal. The bug: the selector saw every row, validation folds included, and kept the 10 columns that best matched the labels it would later be graded on.

Question — why don't five fairly consistent folds expose the leak?

All five validation folds took part in the selection, so their agreement is not evidence of anything. Four of the five sit between 0.92 and 0.98, which is exactly what a trustworthy result looks like. Cross-validation did not fail; it was lied to. It faithfully measured a procedure that had already peeked. This is among the most common serious errors in applied machine learning, published work included.

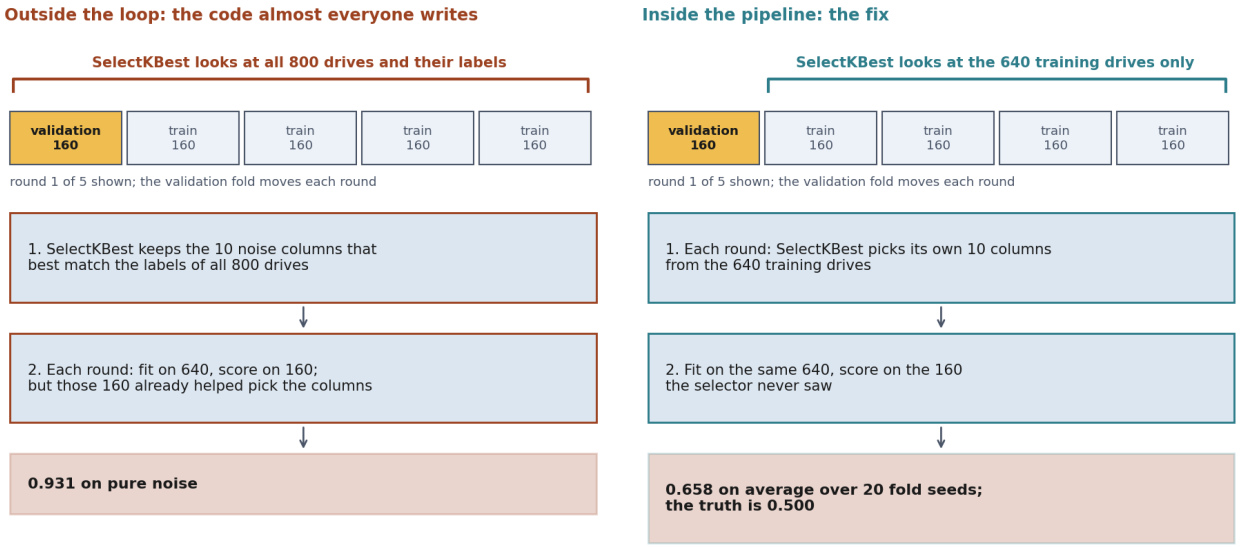
Slide 46 — Fix it, and look what happens

The fix is one `Pipeline`: the selector becomes its first step, so in every round it is refitted on that round's 640 training drives only. Over 20 fold seeds the AUC becomes **0.658** on average, from 0.576 to 0.759. Better. And not 0.500.

Question — why 20 seeds?

The seeds change only how the 800 drives are cut into folds; data, labels and columns stay identical. Each is a full 5-fold cross-validation, and all twenty land above 0.5, so it is not the bad luck of one cut.

Slide 47 — The fix, drawn: the selector sees only training rows



The bracket shows which drives `SelectKBest` looks at. Left, all 800, the validation fold included: 0.931. Right, only the 640 training drives of each round: 0.658.

Slides 44 to 46 in one picture. Same data, same 10 columns kept, same model; the only difference is where the selector sits. Outside the loop, it runs once on all 800 drives and their labels, so in every round the 160 drives being scored have already helped pick the columns. Inside the pipeline, `cross_val_score` refits it in every round on the training drives only, and the 160 drives scored have no say in the choice. What is left, 0.658 against a truth of 0.500, is the subject of the next slide.

Slide 48 — Why it does not return to 0.5

With 2,000 columns and 800 rows, some columns match the label **by accident**, and that accident belongs to the **sample**: it is present in every subset of it, so in every fold. A selector fitted honestly on four fifths finds those columns, and they still work on the remaining fifth. What helps: a test set set aside before any search; fewer candidates; more rows.

Question — why does each of the three help?

None of them is another cross-validation, which only reshuffles the same 800 rows where the accident lives.

- **A test set set aside first** *exposes* the accident. In new rows the chosen columns' correlation with the label is a fresh coin toss, zero on average. The test set has chance matches of its own, but in other columns, which the model does not use. Atom 14 scores the honest pipeline on 40 fresh fleets: they average 0.509.
- **Fewer candidates** *shrink* it: the luckiest of 2,000 columns is luckier than the luckiest of 20, as the best of 2,000 lottery tickets beats the best of 20.
- **More rows** *shrink* it too, and what counts is the rarer class: a noise column's mean over 29 failures wobbles by 0.19, over 290 by 0.06.

Question — is the honest pipeline still leaking, then?

No: 0.658 is not a leak between folds, it is the sample's own chance correlations. The statement to remember: cross-validation protects you from leaking between folds; it cannot protect you from having searched a large space of possibilities on a small sample.

Slide 49 — GridSearchCV: try all, keep the best

The honest pipeline still has two settings to choose: k, the number of columns the selector keeps (2, 5, 10, 20, 40), and C (0.01, 0.1, 1, 10, 100). Five values of k times five of C make **25 pairs**; each is scored by 5-fold cross-validation, $25 \times 5 =$ **125 fits**. The highest mean AUC wins: `best_params_` (k = 20, C = 10), and `best_score_` is that mean. Then the winner is refitted on all 800, fit 126, and not scored again. It is slide 10's left recipe, written once for you.

Question — what exactly is best_score_?

Each pair runs its own 5-fold cross-validation and gets five AUCs, one per round, and their mean. Twenty-five rows like this; four of them:

Pair	Round 1	Round 2	Round 3	Round 4	Round 5	Mean = sum ÷ 5
$k = 20$, $C = 10$, the winner	0.750	0.754	0.879	0.740	0.877	$4.000 \div 5 =$ 0.800
$k = 20$, $C = 1$, second	0.732	0.748	0.899	0.728	0.884	0.798
$k = 20$, $C = 100$, third	0.751	0.741	0.876	0.744	0.879	0.798
$k = 2$, $C = 0.01$, last	0.717	0.641	0.712	0.610	0.739	0.684

`best_params_` names the row with the highest mean; `best_score_` is that mean, 0.800. Its five AUCs come from five models trained during the search; the model refitted on all 800 comes afterwards, and nobody scores it: scored on the drives it trained on, it would give a training score. The top three are nearly tied, the winner wins by a hair, as on slide 12.

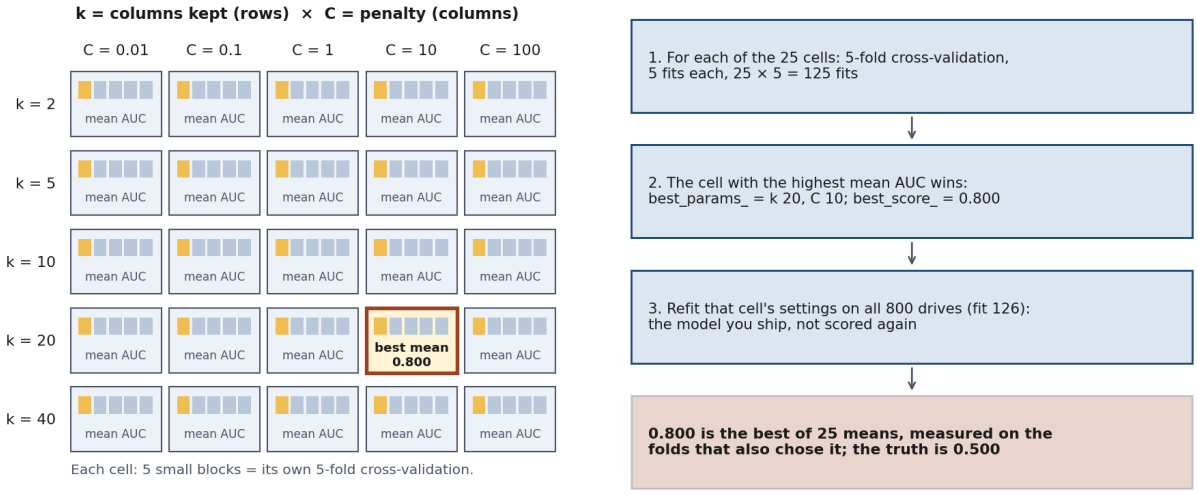
Question — 25 values of C times 5 columns, 125 fits?

The 125 is right and the reasoning is not: five values of k times five of C make the 25 pairs, each costs five fits, and the refit is fit 126.

Question — is k in the grid the number of folds?

The natural reading, since k has meant the folds since slide 13. Here it is the selector's parameter, the number of columns kept; the folds are fixed at 5.

Slide 50 — GridSearchCV, drawn: 25 cells, 5 folds each



Rows are k, the columns kept; columns are C, the penalty. Each cell holds its own 5-fold cross-validation. The outlined cell, k = 20 and C = 10, has the best mean, 0.800.

The previous slide's procedure as a grid. Each of the 25 cells is one pair of settings, and the five small blocks inside it are its own 5-fold cross-validation: five fits, five round AUCs, one mean, so 125 fits in all. The cell with the highest mean wins; its settings are refitted on all 800 drives, fit 126, and that refitted model is the one you ship, not scored again. The box at the bottom right is the catch: 0.800 was measured on the same folds that chose the winner, and on this data the truth is 0.500.

Slide 51 — The best of 25 scores is inflated: 0.800 for a worthless model

The search on the signal-free table: best of the 25 candidates 0.800, average 0.726, worst 0.684, the truth 0.500. The luck of the winner is best minus average, $0.800 - 0.726 = 0.074$.

Question — so taking the best of the 25 gives an inflated result?

Yes, and twice over. The *average* candidate, 0.726, is already far above 0.5: that is slide 48's accident of the sample, present even with the selection inside the pipeline. The *best* adds its own luck on top: the maximum of 25 noisy numbers is pushed upward even when they all measure the same thing. So 0.800 is not the performance of the chosen configuration; it is the performance of whichever configuration got luckiest. Choosing on the data you then report is the same problem as the selection of slide 44.

Worked example — the best of five worthless candidates

Five candidates whose true AUC is 0.50, each measured once with some noise: 0.48, 0.55, 0.51, 0.46, 0.53. Their average is $(0.48 + 0.55 + 0.51 + 0.46 + 0.53) \div 5 = 0.506$, close to the truth. Their maximum is 0.55, and it is the number a search reports. Nothing was done dishonestly; the maximum of noisy numbers is pushed up by the noise, by 0.044 here. With 25 candidates instead of 5 the luckiest is luckier still.

Slide 52 — Nested cross-validation: slide 10's recipe, five times

In `GridSearchCV` the same folds both choose and score, so `best_score_` is the luckiest of 25. Nested cross-validation separates the two jobs, and the bridge is slide 10: set a test set aside, choose on the rest, refit, score the test once. Here the test set is 160 drives; the choosing is the whole search, run on the other 640 with its own five inner folds; the winner is refitted on the 640 and scored once on the 160. Then the 160 rotate, five times, so every drive is tested once. The mean of the five outer scores is **0.670**.

Question — how does it work, step by step?

Outer round 1, with notebook 3's seeds:

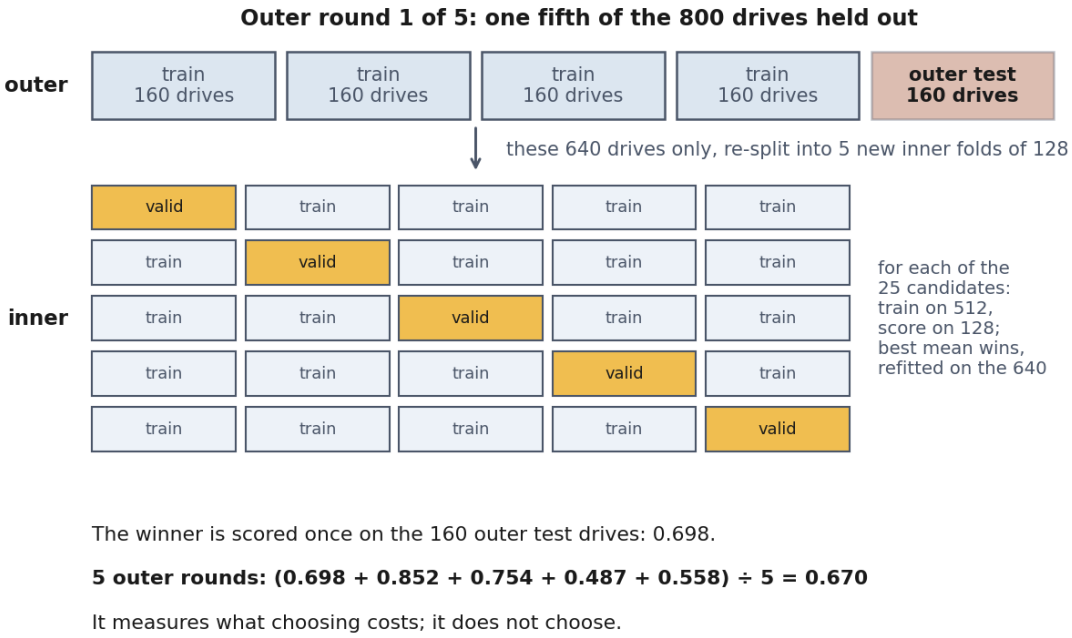
1. Set aside 160 drives as the outer test. Nothing that follows sees them.
2. The other 640 pass to the search as one block and are re-split into five new inner folds of 128. For each of the 25 candidates, five inner rounds: train on 512, score on 128; 125 fits.
3. The best inner mean wins. That is this search's own `best_score_`, and it is thrown away.
4. The winner is refitted on the 640: fit 126.
5. That one model is scored once on the 160: 0.698.

Rounds 2 to 5 repeat everything with another outer fold, and possibly another winner: 0.852, 0.754, 0.487 and 0.558. The nested figure is $(0.698 + 0.852 + 0.754 + 0.487 + 0.558) \div 5 = 3.349 \div 5 = 0.670$, and the whole thing costs $5 \times 126 = 630$ fits.

Question — why is it more honest?

Because choosing, on the 640, and scoring, on the 160, use different drives. The winner's luck stays inside the 640 and does not travel to the 160.

Slide 53 — Nested cross-validation: it measures what choosing costs



Outer round 1 of 5. Look at the arrow: the 640 are re-split into five new inner folds.

The previous slide, drawn. In code it is one line, `cross_val_score(GridSearchCV(honest, grid, cv=inner), noise, y, cv=outer)`: the search goes where the model usually goes. Against `best_score_`'s 0.800, the nested 0.670 gives the optimism of the search: $0.800 - 0.670 = 0.130$, larger than most differences anyone reports between competing methods.

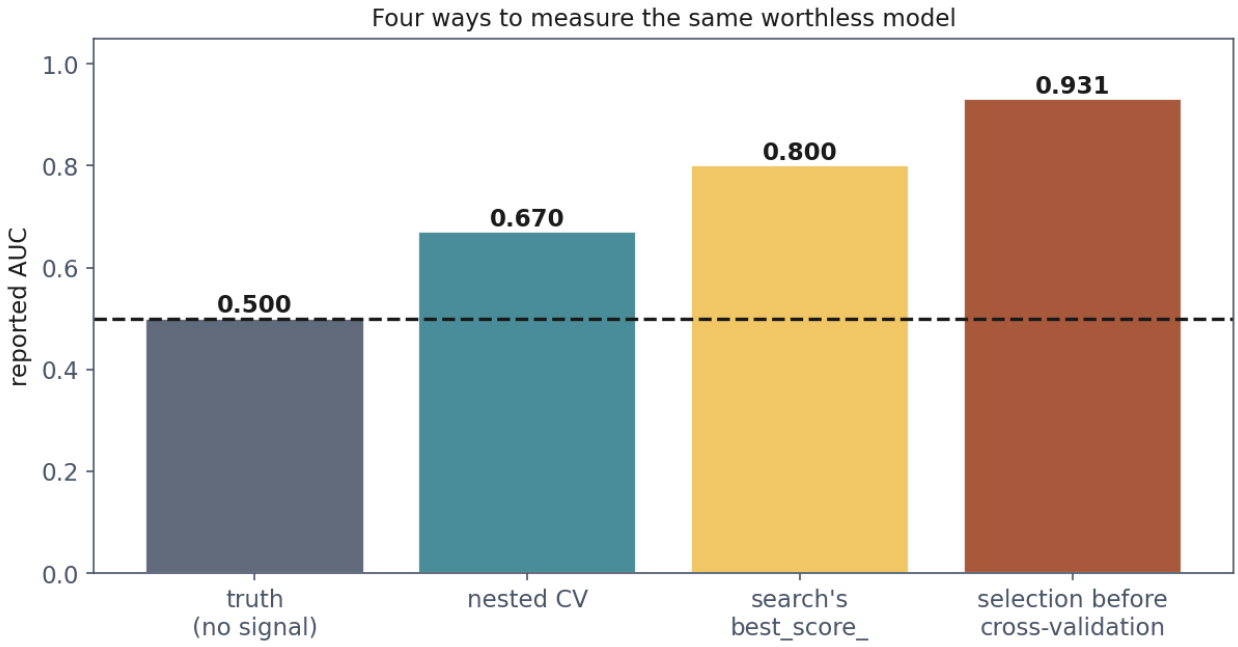
Question — are the inner folds the four outer blocks?

No. The 640 are passed as one block and re-split into five new folds of 128; the figure stacks four outer blocks above five inner ones, which invites the confusion.

Question — is 0.670 the score of the model I ship?

No. The five outer rounds may pick different winners (here they do), and the model you would ship, chosen and refitted on all 800, is none of them. Nested cross-validation does not choose hyperparameters; it measures what choosing costs. Choose afterwards on all the data, and report the nested figure. It is honest but itself noisy: its five outer scores run from 0.487 to 0.852.

Slide 54 — Each piece of discipline costs score: 0.931, 0.800, 0.670, 0.500



Four ways to measure the same data, which contain no signal at all.

The whole last hour on one axis. 0.931: selection before cross-validation. 0.800: the search's `best_score_`. 0.670: nested cross-validation. 0.500: the truth. Nobody in this picture wrote dishonest code; the difference is entirely which choices were made while looking at which data.

Question — why does nested cross-validation come closer to 0.5?

One rule reads the ladder: *a score is honest to the extent that the rows that measure it took part in no decision.* Each step to the left removes one decision taken while looking at the scoring rows.

Step	What the scoring rows had a hand in	As an exam
0.931	choosing the **columns**: the selector saw their labels	the questions and the answers seen beforehand
0.800	choosing the **configuration**: the same folds choose and score	25 mock exams, the best mark reported
0.670	**nothing**: every choice on the 640, every score on the 160	an exam never seen, after preparing on mock exams

Question — then why 0.670 and not 0.500?

The 0.17 that remains is slide 48's accident of the sample. The columns that match the label by chance match it in all 800 drives, the 160 outer test drives included: nested cross-validation separated choosing from scoring, but inside the same sample. The exam never seen still comes from the same question bank. Only an exam from another school, new data, brings the number back to

0.5: a test set set aside before any search exposes the accident, while fewer candidates and more rows shrink it.

Part V — The debt, and what to report

Slide 55 — The debt, paid: the honest threshold costs 1,400 EUR more

Lesson 4's threshold, done properly, on slide 6's 800 drives: fit on 450, choose the threshold on 150, report once on 200 with 7 failures. One model, fitted once on the 450, serves all three rows; they differ only in the threshold, and every bill is counted on the same 200 test drives, at 2,600 EUR per missed failure and 140 EUR per false alarm.

Threshold	How it was chosen	Bill on the 200 test drives
0.50	nothing chosen, the default	$4 \times 2{,}600 + 1 \times 140 = 10{,}540$ EUR
0.050	the cheapest of 200 thresholds, tried on the 150 validation drives	$1 \times 2{,}600 + 15 \times 140 = 4{,}700$ EUR
0.171	the same sweep, on the 200 test drives (lesson 4's way)	$1 \times 2{,}600 + 5 \times 140 = 3{,}300$ EUR

Question — the honest threshold costs only slightly more than the cheating one?

Not slightly: $4{,}700 - 3{,}300 = 1{,}400$ EUR more, 42% more. Both thresholds catch 6 of the 7 failures; the difference is ten false alarms that this particular test set happened to let 0.171 avoid. And the honest number *must* be worse: paying the debt removes the luck the cheating number had borrowed from the test set. 3,300 describes a threshold tuned to these 200 drives, with no reason to survive the next 200; 4,700 is the honest estimate of what happens next. The slip "the debt is paid, so the number improves" is natural, since doing things properly ought to be rewarded.

Question — what else does the table say?

That choosing a threshold at all matters: the default 0.5 misses 4 of the 7 failures and costs 10,540. And that the validation set chose 0.050, almost exactly lesson 4's $t^* = 140 \div 2{,}740 = 0.051$, the threshold the costs alone gave before any data were seen; with five failures in the validation set part of that is luck, but the formula was already the honest answer. The euros are not comparable with lesson 4's, which were billed on a much larger test set. And honesty has a cost: the model is fitted on 450 drives instead of 600, which is the argument for choosing by cross-validation on the 600 and keeping the test set whole.

Worked example — three thresholds, six validation drives, six test drives

One model has given probabilities to six validation drives, two of them failing, and to six test drives, two failing. A drive is flagged when its probability is at least the threshold; a missed failure costs 2,600 EUR, a false alarm 140.

threshold	bill on the validation drives (EUR)	bill on the test drives (EUR)
0.05	$0 \times 2{,}600 + 3 \times 140 = 420$	$0 \times 2{,}600 + 3 \times 140 = 420$
0.10	$1 \times 2{,}600 + 2 \times 140 = 2{,}880$	$0 \times 2{,}600 + 1 \times 140 = 140$
0.50	$1 \times 2{,}600 + 0 \times 140 = 2{,}600$	$1 \times 2{,}600 + 0 \times 140 = 2{,}600$

On the validation drives the cheapest threshold is 0.05: that is the choice, and the test set then reports 420 EUR. Had the threshold been chosen on the test drives themselves, 0.10 would have won with 140 EUR: cheaper, and not reportable, because it was picked by looking at the bill being reported.

Slide 56 — Three sets, three jobs

The principle of the whole lesson. **Train** fits the parameters. **Validation**, a held-out set or the cross-validation folds, makes every choice: model, hyperparameters, features, threshold. **Test** reports, once. Change anything after looking at the test set and it has become a validation set.

Question — can I use the test score to decide which model to submit?

Then it is no longer a test score: you no longer have a test set at all. Everyone looks twice sometimes; the requirement is to **say so in the report**, or to get fresh data. A stated weakness costs a little credibility, an unstated one all of it.

Slide 57 — A fixed seed buys repeatability, not stability (optional)

Thirty seeds of the train/test split, same model: from 0.913 to 1.000. Every one of the thirty runs is perfectly reproducible, and they disagree by nearly nine points. "AUC 1.000 (random_state=3)" is reproducible and misleading. Report the seed **and** the spread: "AUC 0.951 ± 0.019 over 5-fold cross-validation, seed 0", slide 14's five round AUCs.

Question — if seed 3 always gives 1.000, isn't that a reproducible result?

As a computation, yes; as a description of the model, no. It is one unusually favourable split, repeated exactly. Reproducibility is necessary and nowhere near sufficient.

Slide 58 — What to fix, and what to write down (optional)

Fix every `random_state` (the split, the folds) and seed your own NumPy draws with `np.random.default_rng(seed)`, so that a rerun gives the same number. Record the library versions, because defaults change between releases; the course container exists for this. And state what cannot be fixed: parallel computations that finish in a different order, graphics processing unit (GPU) non-determinism.

Question — random_state for scikit-learn, a seeded generator for my own draws?

Yes. `random_state` goes on every call that draws, not once globally; any integer works, what matters is writing it down. In the course's scikit-learn version, pass it an integer, not a NumPy generator.

Slide 59 — Notebook 3, live

The map of notebook 3: 0.931 with the selection before cross-validation; 0.658 with the selector inside the pipeline; 0.800 as the search's `best_score_`; 0.670 from nested cross-validation; the threshold, 4,700 EUR against 10,540; and 30 seeds from 0.913 to 1.000. On 6 November atoms 11, 12 and 13 run in its place.

Slide 60 — What to put in a report

Four items, one for each failure of the lesson. The metric, its **spread** and how it was estimated, since a number without a spread hides the lottery. How many **positives** the test set held, since that sets how wide the spread is. Every seed and every library version, for reproducibility. Which choices were made on which data, since an unstated selection hides the optimism.

Question — where does the spread come from, for a cross-validation?

From the standard deviation of the fold scores: `scores = cross_val_score(...)`, then `scores.mean()` and `scores.std()`. In the report: "AUC 0.951 ± 0.019 (standard deviation over 5 folds), stratified 5-fold, seed 0". Not $s/\sqrt{5}$, and not a confidence interval: slide 17 showed the real spread is twice that. A single test set gives one number with no spread of its own, which is why the report also states the positives.

Slide 61 — What to take away

A score is a measurement: 0.885 to 1.000, the number the lesson opened with. Choosing on one split chooses noise. A high training error points to bias, a gap to variance. Choose on validation, report on test once. And three things cross-validation cannot remove: the sample's chance correlations (0.658 on pure noise), the luck of choosing (0.800 reported, 0.670 nested), and dependent rows unless they are split with `GroupKFold`.

Question — what do all the problems of the lesson have in common?

Two shapes. Either a decision used information that should not have been available when the reported score was produced: the selector, the search, the threshold on the test set. Or the score is simply a noisy measurement even when the procedure is legitimate: the 200 splits, the 7 failures. The habit to keep: before believing a score, ask how many positives are in the test set, and which choices were made while looking at which data.

Slide 62 — Homework

Exercise 5, discussed at the start of Friday 20 November: a notebook that reports a result too good to be true. Find out why, isolate each problem, produce the honest number, and explain what it means to the people who will use it.

Question — is it only about fixing the colleague's code?

No. It asks to identify and rank the concerns, test each one in isolation, produce a defensible number with its spread, and explain in plain words what it does and does not promise, with the reasoning written in markdown cells beside the evidence. There is more than one problem planted, and the honest figure will be much lower than the one you start with: that is the expected outcome. At the exam, what counts is the diagnosis.